

尚硅谷嵌入式项目之畜牧定位器

(作者：尚硅谷研究院)

版本：V1.0.1

第 1 章 项目需求

1.1 项目概述

该畜牧定位器是智能农场项目的子项目，目的在于定位农场中的畜牧牲畜。由于牲畜的运动范围有限(农场中)，故定位频率不用太高，一般小时级别的定位频率即可满足需求。

该定位器也带有计步功能，可以记录牲畜的运动步数。

1.2 功能描述

1.2.1 定位器

牲畜身上绑定定位器，主控芯片为 **STM32F103C8T6**，定位器开发板上的主要功能模块是：**LoRa 模块**，**NB-IoT 模块**，**GPS 模块**，**计步模块**。

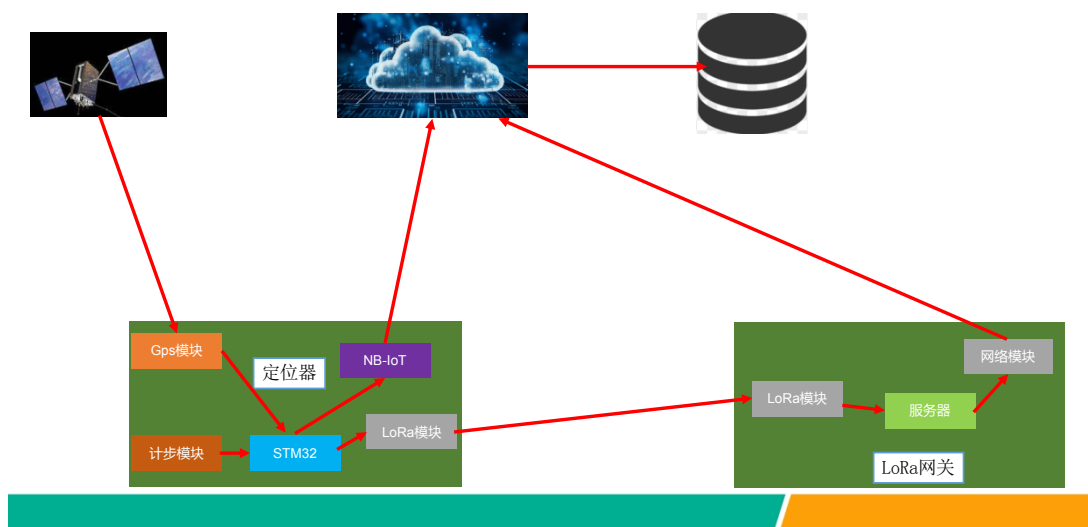
牲畜通过 GPS 模块获取自身的 GPS 定位信息，通过计步模块获取运动步数，然后通过 NB-IoT 模块把这些发送到云端服务器。云端服务器收到这些信息后，会自动存储到数据库。其他应用会从数据库读取这些信息，然后再根据需要进行各种分析。

如果 NB-IoT 出现无网络的情况，则无法发送到云端服务器。这时可以通过 LoRa 发送到 LoRa 网关，再由网关统一把收到的定位和计步信息发送到云端服务器。

1.2.2 LoRa 接收网关

只负责接收 LoRa 发来的数据，然后通过网络发送到云端服务器。LoRa 接收网关连接网络可以有有线也可以是 Wifi 无线连接。

1.3 系统总体设计



第 2 章 硬件架构

2.1 硬件选型

2.1.1 定位器

主控芯片：STM32F103C8T6。

LoRa 模块： LoRa 芯片:LLCC68。

GPS 模块： AT6558R-5N32

计步模块： DS3553

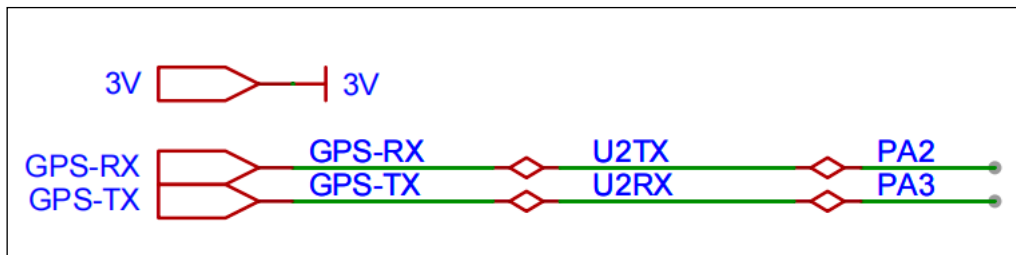
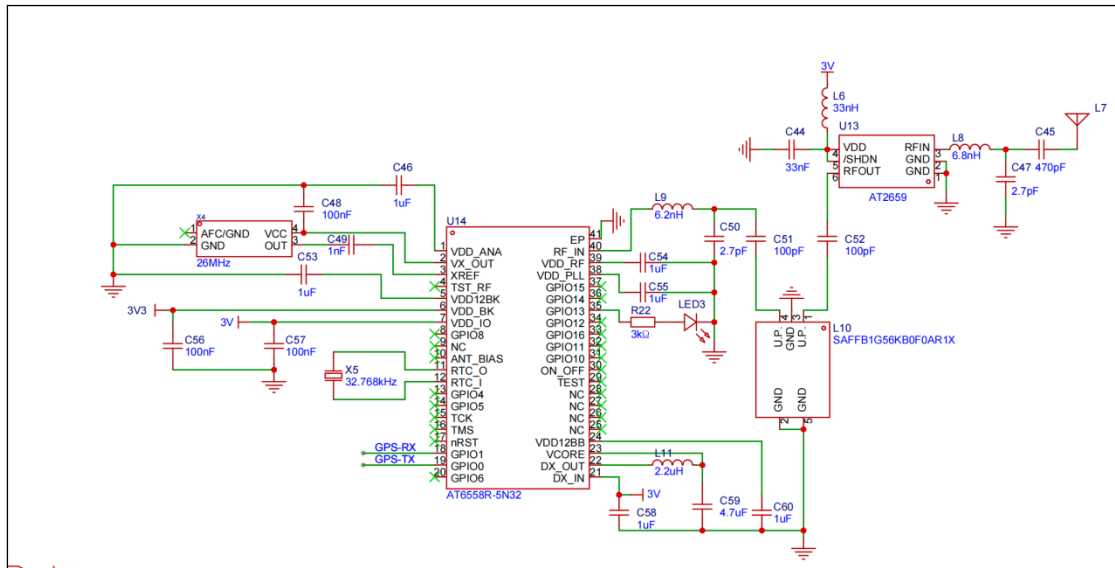
2.1.2 LoRa 网关

普通的服务器即可,能连接网络,能接收 LoRa。暂时使用我们的 STM32 开发板来做 LoRa 网关。

2.2 GPS 模块

AT6558R, GPS 模块接收到 GPS 信号后, 通过串口与 STM32 进行通信。芯片支持多种卫星导航系统, 包括中国的北斗卫星导航系统 BDS, 美国的 GPS, 俄罗斯的 GLONASS, 并实现多系统联合定位。

2.2.1 原理图



2.2.2 接收到的卫星数据格式

```
$GNGGA,070822.000,4006.81888,N,11621.89413,E,1,05,25.5,30.2,M,-9.6,M,,*58
$GNGLL,4006.81888,N,11621.89413,E,070822.000,A,A*49
$GNGSA,A,3,05,12,25,,,,,,,,,25.5,25.5,25.5,1*02
$GNGSA,A,3,10,13,,,,,,,,,25.5,25.5,25.5,4*05
$GPGSV,3,1,11,05,54,263,24,06,32,098,,09,20,043,,11,68,061,,0*61
$GPGSV,3,2,11,12,14,237,17,13,23,176,,15,05,204,,19,08,152,,0*60
$GPGSV,3,3,11,20,75,327,,25,13,265,23,29,28,314,,0*51
$BDGSV,2,1,05,08,64,301,,10,47,216,15,13,56,283,10,27,59,133,,0*78
$BDGSV,2,2,05,38,72,326,,0*48
$GNRMC,070822.000,A,4006.81888,N,11621.89413,E,0.81,359.02,020624,,A,V*02
$GNVTG,359.02,T,,M,0.81,N,1.51,K,A*22
$GNZDA,070822.000,02,06,2024,00,00*47
$GPTXT,01,01,01,ANTENNA OPEN*25
```

2.2.3 参考资料



AT6558R.pdf



接收机协议规范.pdf

2.3 NB-IoT 模块

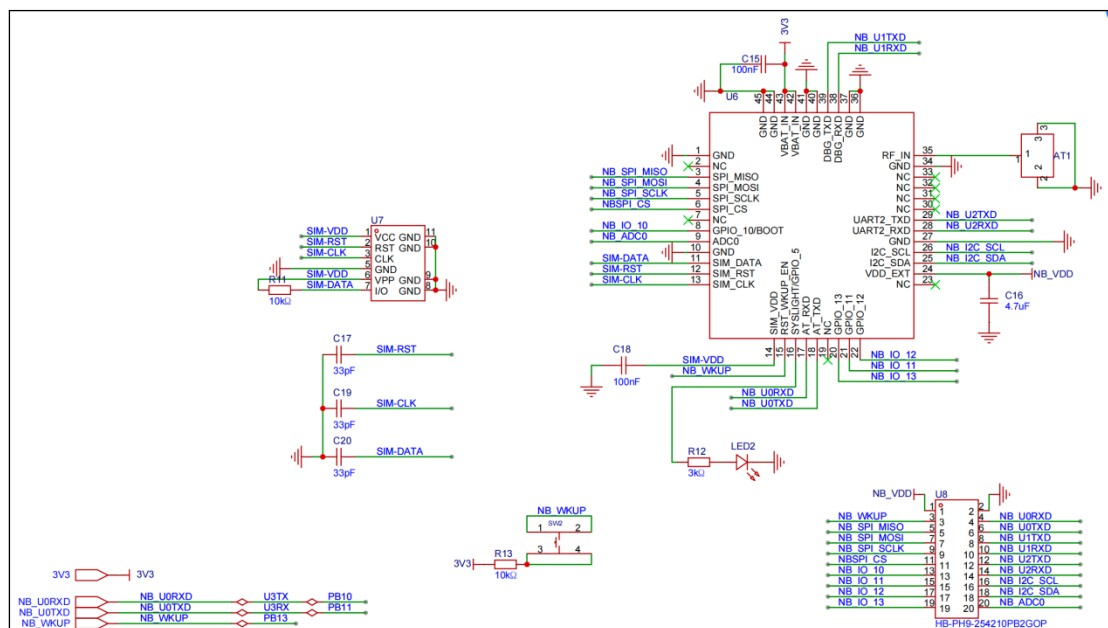
NB-IoT（Narrowband Internet of Things）是由 3GPP（第三代合作伙伴计划）制定的一种窄带物联网（IoT）标准。它专为低功耗广域网（LPWAN）设计，旨在通过**移动蜂窝网络**提供广覆盖、低功耗、低成本连接。

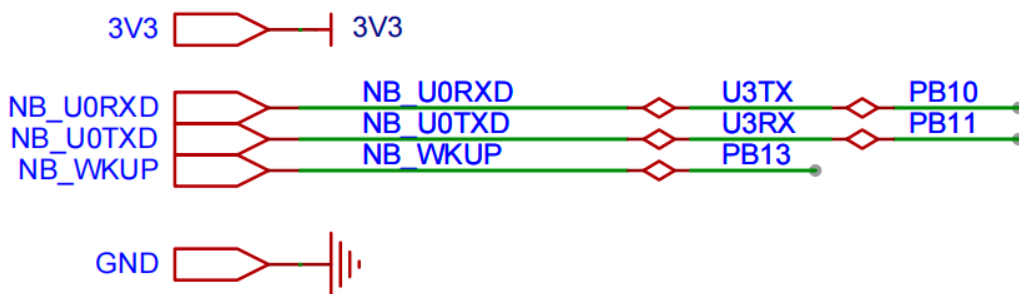
QS-100 是一款基于芯翼 XY1100 的工业级 NB-IoT 无线通信模组，具有高性能、低功耗、多频段等多个产品优势，协议上完全兼容 R13 和 R14，软件升级支持 R15，内嵌 TCP、UDP、MQTT、COAP 等多种通信协议栈，支持中国移动 OneNET，中国联通云和中国电信 IoT 物联网云平台，方便客户应用。

QS-100 模组采用 44pins 的 LGA 封装，其尺寸为 15.8mm*17.7mm*2.2mm，通过焊盘内嵌于各类数传产品应用中，同时模组提供了丰富的外围接口供客户主板和模组通信，便于客户嵌入到各种产品中，能够满足物联网应用需求，包括电表、燃气表、水表、烟感、路灯、井盖、消防栓、农业和环境监测等。

STM32 **通过串口发送 AT 指令**来操作 QS-100

2.3.1 原理图





2.3.2 参考资料


[QS-100硬件设计手册v1.1.pdf](#)

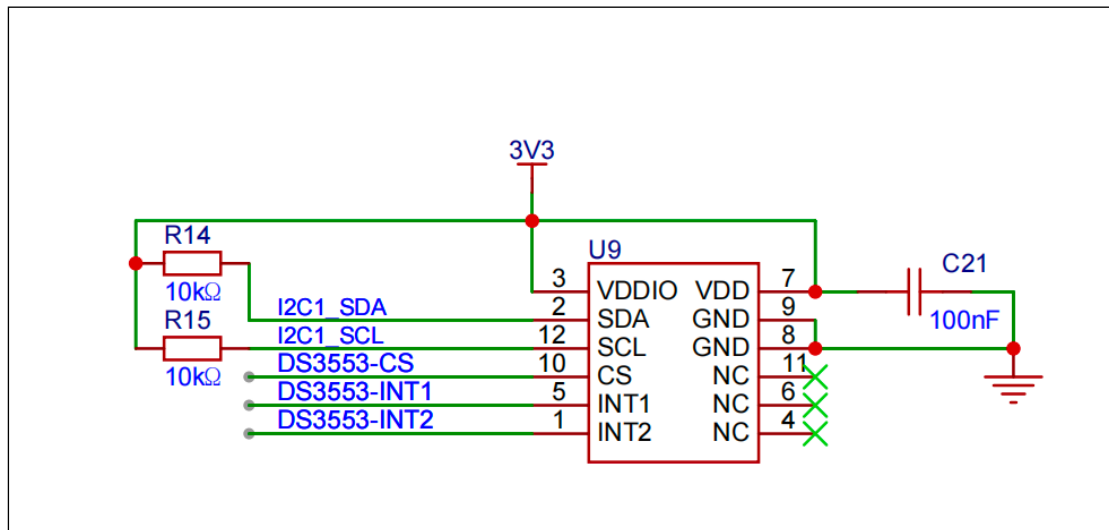
[QS-100模块AT命令手册_V1.0.pdf](#)

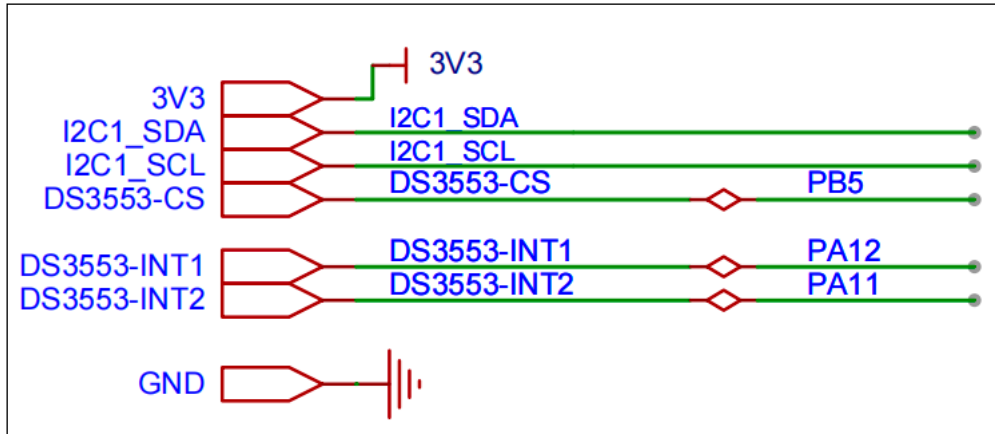
2.4 计步模块

智能计步传感器 DS3553 集成了一个高精度、高分辨率的明暗加速度传感器, 和一个 8 位低功耗嵌入式微控器的专用架构。由于其低功耗、高精度的性能优势, 被广泛的应用于各种智能穿戴式计步产品, 包括手环、计步鞋、计步器、运动耳机等。

通过 I2C 与 STM32 进行通信。

2.4.1 原理图





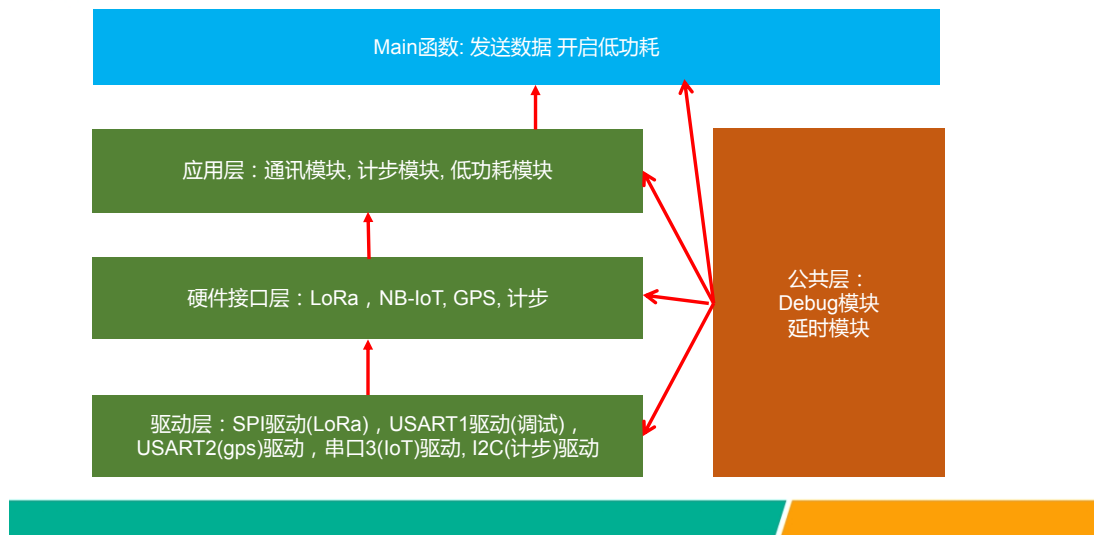
2.4.2 参考资料



ds3553.pdf

第3章 软件架构

3.1 定位器软件架构



3.2 LoRa 网关架构



第 4 章 背景知识

4.1 GPS 定位原理

设 GPS 接收机的坐标信息是 (x, y, z, t) 。卫星 1 的坐标信息是 (x_1, y_1, z_1, t_1) 。卫星 2 的坐标信息是 (x_2, y_2, z_2, t_2) 。卫星 3 的坐标信息是 (x_3, y_3, z_3, t_3) 。卫星 4 的坐标信息是 (x_4, y_4, z_4, t_4) 。

其中 t_1, t_2, t_3, t_4 是各个卫星原子钟的时间。卫星和 GPS 接收机之间的距离使用欧几里得距离 $\sqrt{(x - x_1)^2 + (y - y_1)^2 + (z - z_1)^2}$ 。同时也等于光速 c 乘以 $(t - t_1)$ 。因为卫星在向 GPS 接收机发送信息时，自身的时间是 t_1 ，信息发送至 GPS 接收机时，GPS 接收机自身的时间是 t ，那么 $c(t - t_1)$ 自然就是卫星 1 和 GPS 接收机的距离。

则会有以下方程组成立：

$$c(t - t_1) = \sqrt{(x - x_1)^2 + (y - y_1)^2 + (z - z_1)^2}$$

$$c(t - t_2) = \sqrt{(x - x_2)^2 + (y - y_2)^2 + (z - z_2)^2}$$

$$c(t - t_3) = \sqrt{(x - x_3)^2 + (y - y_3)^2 + (z - z_3)^2}$$

$$c(t - t_4) = \sqrt{(x - x_4)^2 + (y - y_4)^2 + (z - z_4)^2}$$

其中 c 是光速。4 个方程可以解出 4 个未知数。

由于要搜索卫星，所以需要在户外。这样才能搜索到卫星，从而计算出准确的定位信息。

4.2 无线传输基本原理

无线电传输信息靠的是电磁波。电磁波的波形可以用幅度，相位和频率来描述。要发射某个频率的电磁波，必须使用和**电磁波波长同样长度的天线**，才能将电磁波发送出去。

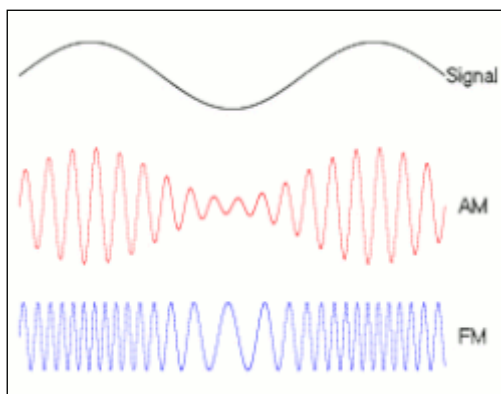
举个例子，人的声音的频率范围在 300Hz~3400Hz。那么根据波长的计算公式：波长 = 光速 ÷ 频率。人的声音的波长的范围是 3^5 米~ 3^6 米。那么为了将人声发送出去，发射天线的长度需要是 30 万米，这显然是不现实的。

所以需要将人的声音搬运到更高的频段，这样需要的发射天线长度就会更短。这个过程叫做**“调制”**。

当接收机接收到电磁波信号时，再将人声信号从高频段搬运到原来的频段。这个过程叫做**“解调”**。

注意：频率越高（波长越小），电磁波越容易被大气、水吸收，也越容易被建筑物遮挡，所以传播距离短。

4.2.1 模拟调制

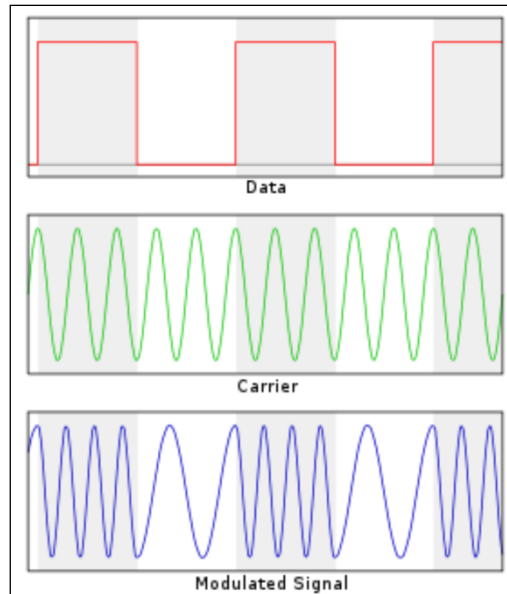


动图中显示了调频（FM）和调幅（AM）的原理。

4.2.2 数字调制

数字调制的目的是如何使用电磁波将 0101 这样的二进制数发送出去。

这里我们举一个 LoRa 模块中使用的 FSK 为例子。FSK（Frequency-shift keying，频移键控）使用不同的频率来表示 0 和 1。



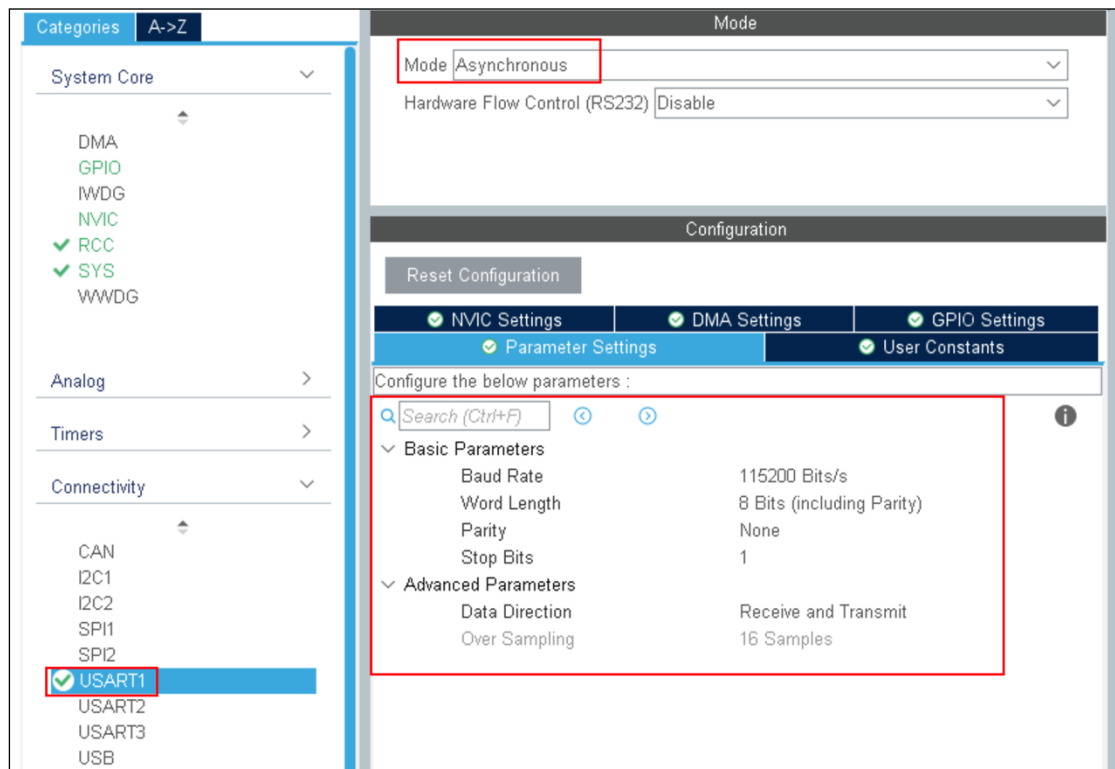
从上到下分别为信号，载波，调制以后的信号。

第 5 章 定位器

5.1 公共层

5.1.1 调试模块

5.1.1.1 CubeMX 配置



5.1.1.2 Debug.h

```
#ifndef __DEBUG_H
#define __DEBUG_H

#include "usart.h"
#include "string.h"
#include "stdarg.h"
#include "stdio.h"

/**
 * 为方便调试
 * 当定义 DEBUG 这个宏之后，所有的输出打印语句会正常打印
 * 如果没有定义这个宏，所有的输出打印语句都会在预处理时被去掉。
 * 当调试代码时：    定义 DEBUG
 * 当 Release 代码时： 不定义 DEBUG
 */
#define DEBUG

#ifdef DEBUG

#define _FILE_NAME (strrchr(__FILE__, '/') ? strrchr(__FILE__, '/') + 1 : __FILE__)
#define FILE_NAME (strrchr(_FILE_NAME, '\\') ? strrchr(_FILE_NAME, '\\') + 1 : _FILE_NAME)

#define debug_init() Debug_Init()
#define debug_printf(format, ...) printf("[%s:%d]" format, FILE_NAME, __LINE__, ##__VA_ARGS__)
#define debug_printfln(format, ...) printf("[%s:%d]" format "\r\n", FILE_NAME, __LINE__, ##__VA_ARGS__)

#else

#define debug_init()
```

```
#define debug_printf(format, ...)
#define debug_printfln(format, ...)

#endif

void Debug_Init(void);

#endif
```

5.1.1.3 Debug.c

```
#include "Debug.h"

void Debug_Init(void)
{
    // 初始化串口
    // MX_USART1_UART_Init();
}

int fputc(int ch, FILE *f)
{
    HAL_UART_Transmit(&huart1, (uint8_t *)&ch, 1, 0xFFFF);
    return ch;
}
```

5.1.2 公共配置文件

在该 h 文件中,定义一些常用的配置,方便管理.

```
#ifndef __CONFIG_H
#define __CONFIG_H
```

```
#include "stm32f1xx.h"

// https://netlab.luatos.com/
// IOT 物联网芯片远程连接的云服务器地址

#define TCP_SERVER_IP "112.125.89.8"

#define TCP_SERVER_PORT 44351

/* 状态类型定义 */
// 使用 CommonStatus 的返回值类型 => 判断当前函数方法是否执行成功
typedef enum
{
    COMMON_OK = 0,
    COMMON_ERROR = 1,
    COMMON_OTHER = 2
} CommonStatus;

/* 通过 IoT 和 LoRa 上传的数据的类型封装 */
typedef struct
{
    // 牛马编号 => 使用物联网卡的 id
    uint8_t uuid[33]; /* 使用 16 进制字符串形式保存 16 * 2 = 32 */

    /* gps 信息 */
    // 0-180 东西 一共 360
    double lon; /* 经度 */
    uint8_t lonDir[2]; /* 经度 方向 E-东 W-西 */

    // 0-90 南北 一共 180
    double lat; /* 纬度 */
    uint8_t latDir[2]; /* 纬度方向 N-北 S-南 */

    // 速度
    double speed; /* 对地速度 单位位节 */

    uint8_t dateTime[21]; /* 定位时间 2024-12-12 11:11:11 */
}
```

```
/* 计步信息 */
uint16_t stepNum; /* 运动的步数 */

/* 存储 json 格式的字符串，用于上传到服务器*/
//{"uid":"51173829395550330676FF32","lat":40.1012077331543,"latDir":"N","lon":1
16.35979461669922,"lonDir":"E","speed":0.56999999284744264,"dateTime":"2025-01-01
06:19:36","stepNum":0}
uint8_t data[256]; /* 缓冲区 */
uint16_t dataLen; /* 缓冲功区存储的实际的数据长度 */
} UploadDataType;

#endif
```

5.1.3 工具函数

5.1.3.1 Tool.h

```
#ifndef __TOOL_H
#define __TOOL_H

#include "stm32f1xx_hal.h"
#include "time.h"
#include "string.h"
#include "stdio.h"
#include "config.h"

void Delay_us(uint16_t us);
void Delay_ms(uint16_t ms);
void Delay_s(uint16_t s);

/* 将 UTC 时间的字符串转换为北京时间的字符串
    2024-10-25 06:07:00 => 2024-10-25 14:07:00
*/
```

```
void UTCTime_to_bjTime(uint8_t *utcTime, uint8_t *bjTime);
```

```
#endif
```

5.1.3.2 Tool.c

```
#include "Tool.h"
```

```
void Delay_us(uint16_t us)
```

```
{
    uint32_t Delay = us * 10;
    while (Delay--)
        ;
}
```

```
void Delay_ms(uint16_t ms)
```

```
{
    HAL_Delay(ms);
}
```

```
void Delay_s(uint16_t s)
```

```
{
    while (s--)
    {
        HAL_Delay(1000);
    }
}
```

```
void UTCTime_to_bjTime(uint8_t *utcTime, uint8_t *bjTime)
```

```
{
    // +8 小时
    /* 1. 解析 UTC 时间 转换为时间戳 */
    struct tm utc;

    int32_t year, month, day, hour, minute, second;

    // utcTime => 2024-10-25 06:07:00
}
```

```
sscanf((char *)utcTime, "%d-%d-%d %d:%d:%d",
        &year, &month, &day, &hour, &minute, &second);

utc.tm_year = year - 1900;
utc.tm_mon = month - 1;
utc.tm_mday = day;
utc.tm_hour = hour;
utc.tm_min = minute;
utc.tm_sec = second;

// 使用 mktime 转换结构体到时间戳
time_t timestamp = mktime(&utc);

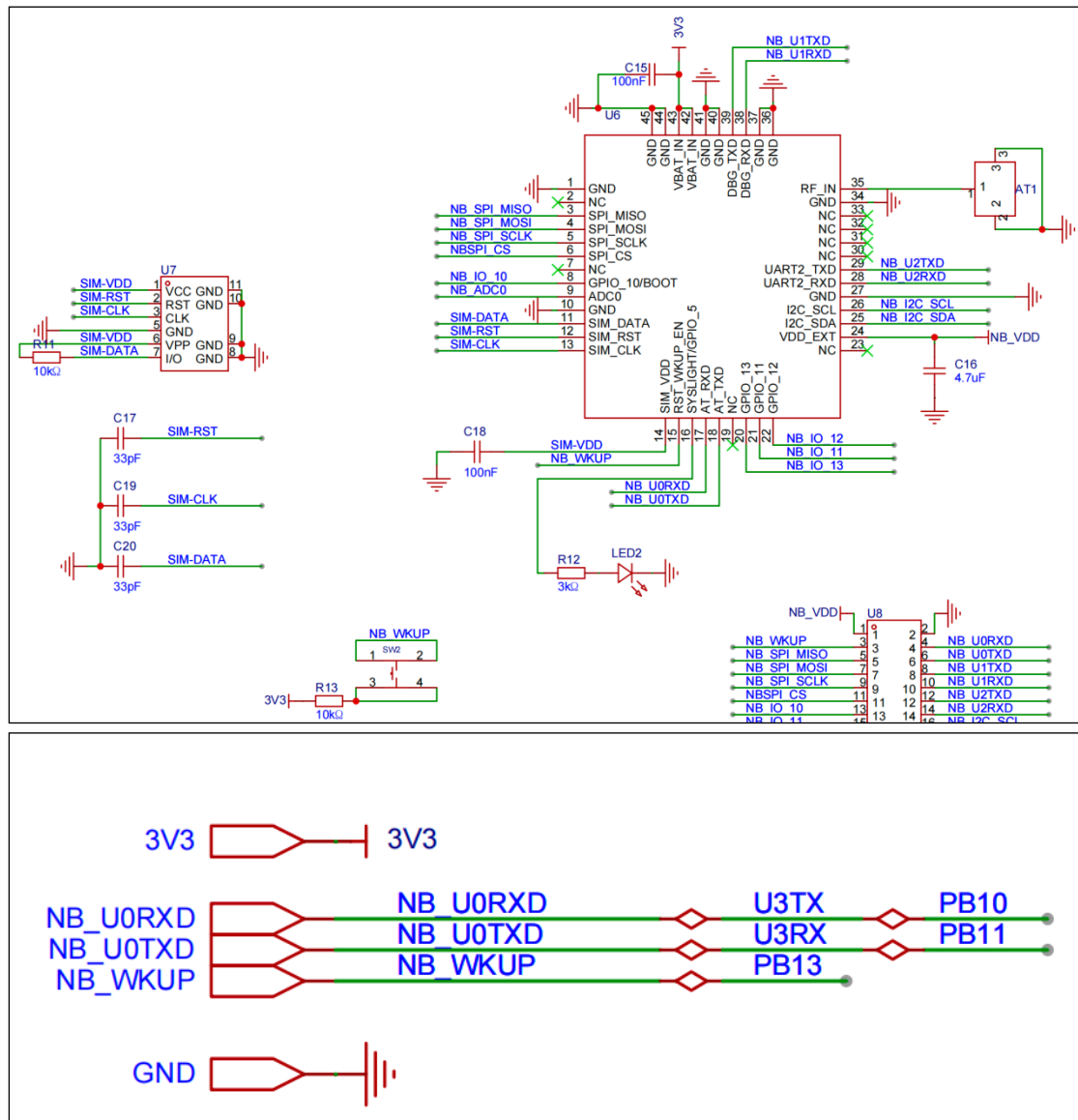
/* 2. 对转换完成的时间戳 +8h */
timestamp += 8 * 3600;

/* 3. 将时间戳转换为北京时间 */
// 使用 localtime 转换时间戳到结构体
struct tm *tm_bj_time = localtime(&timestamp);
sprintf((char *)bjTime, "%04d-%02d-%02d %02d:%02d:%02d",
        tm_bj_time->tm_year + 1900,
        tm_bj_time->tm_mon + 1,
        tm_bj_time->tm_mday,
        tm_bj_time->tm_hour,
        tm_bj_time->tm_min,
        tm_bj_time->tm_sec);
}
```

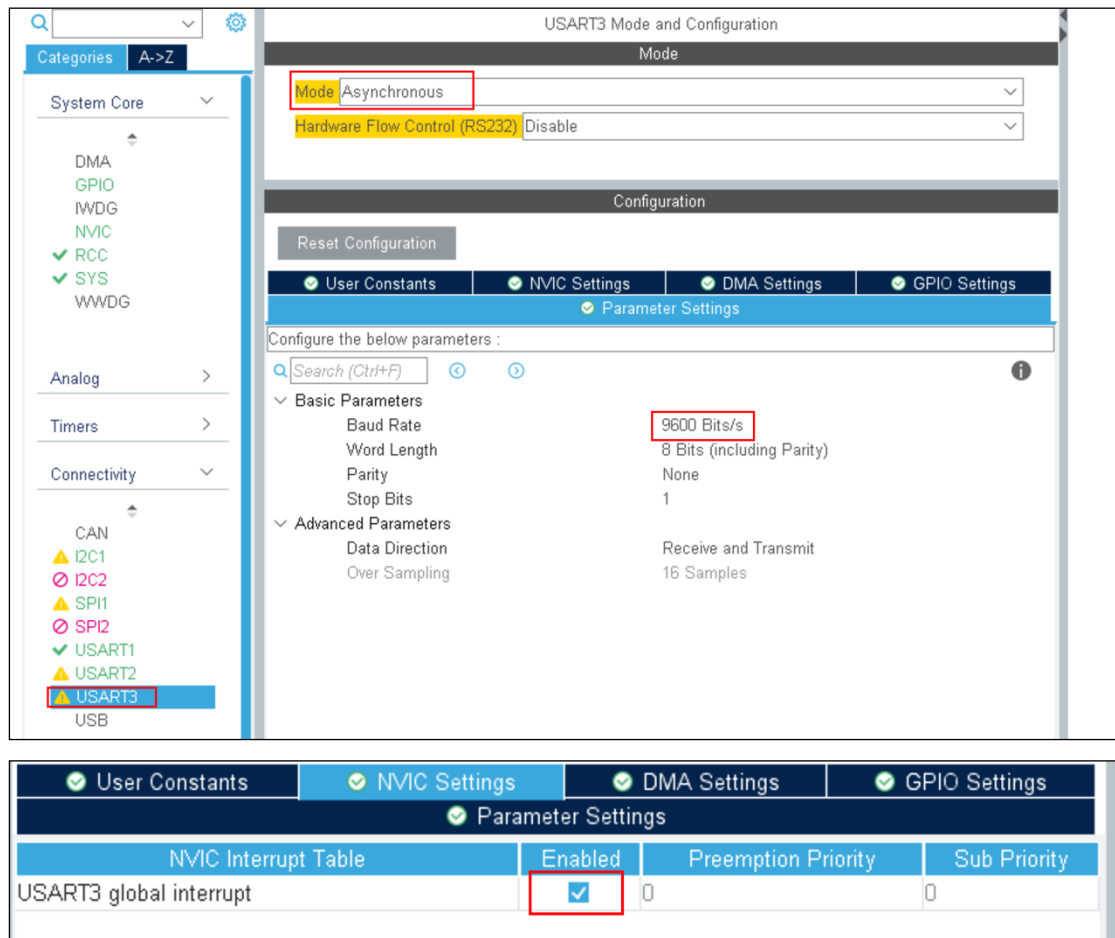
5.2 通信模块

5.2.1 CubeMx 配置

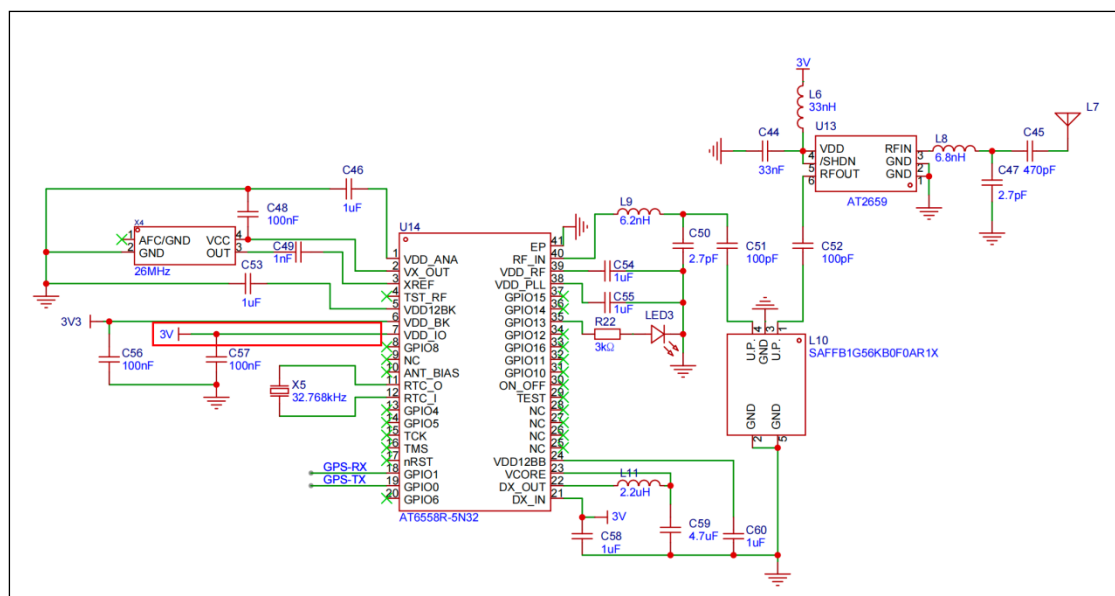
5.2.1.1 NB-IoT 模块硬件连接电路

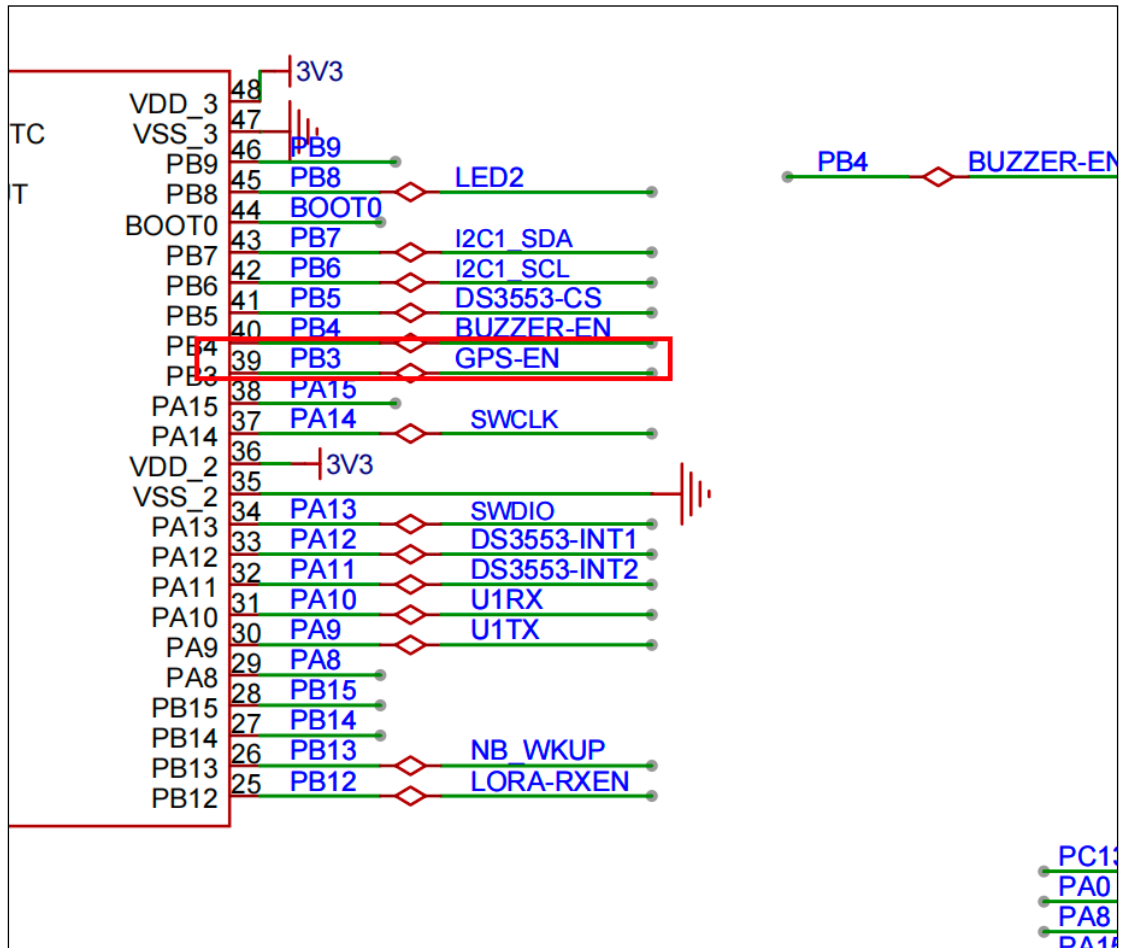
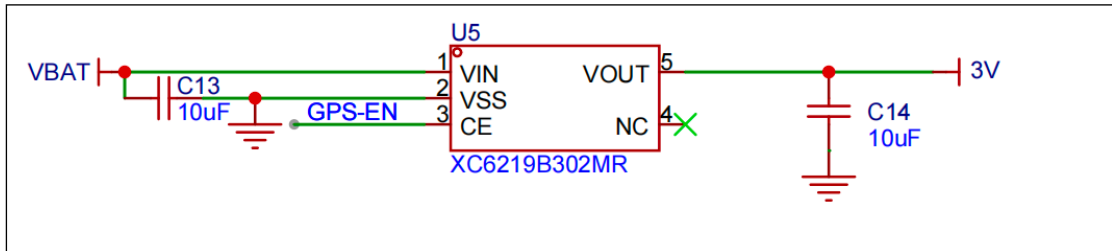
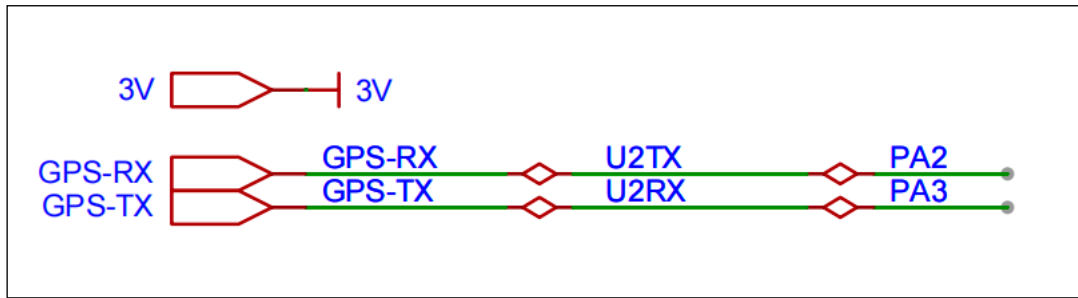


5.2.1.2 NB-IoT 模块硬件 CubeMX 设置

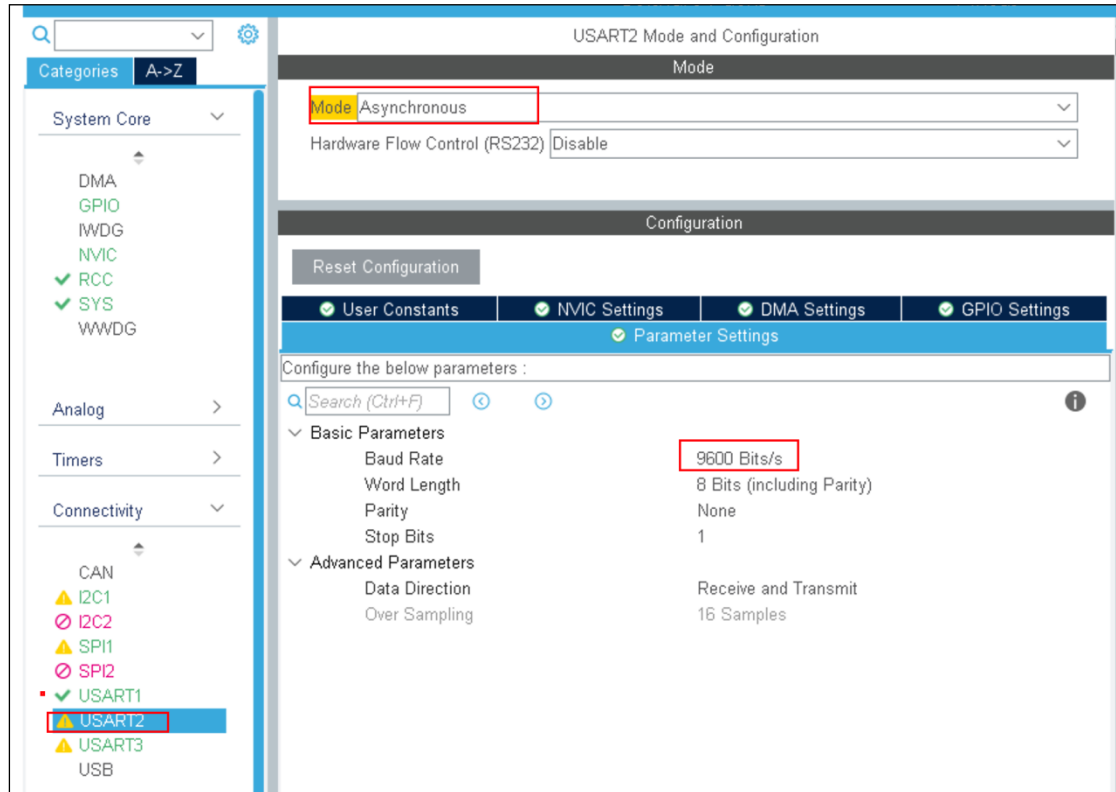


5.2.1.3 GPS 模块硬件连接电路





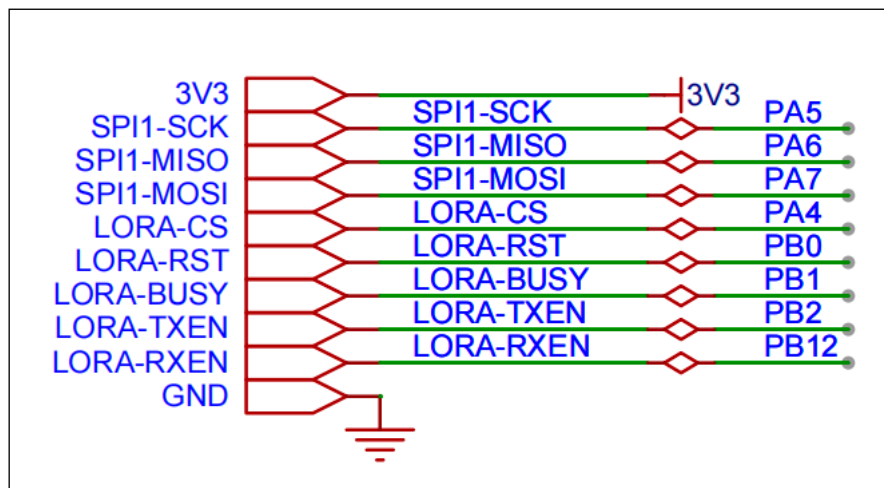
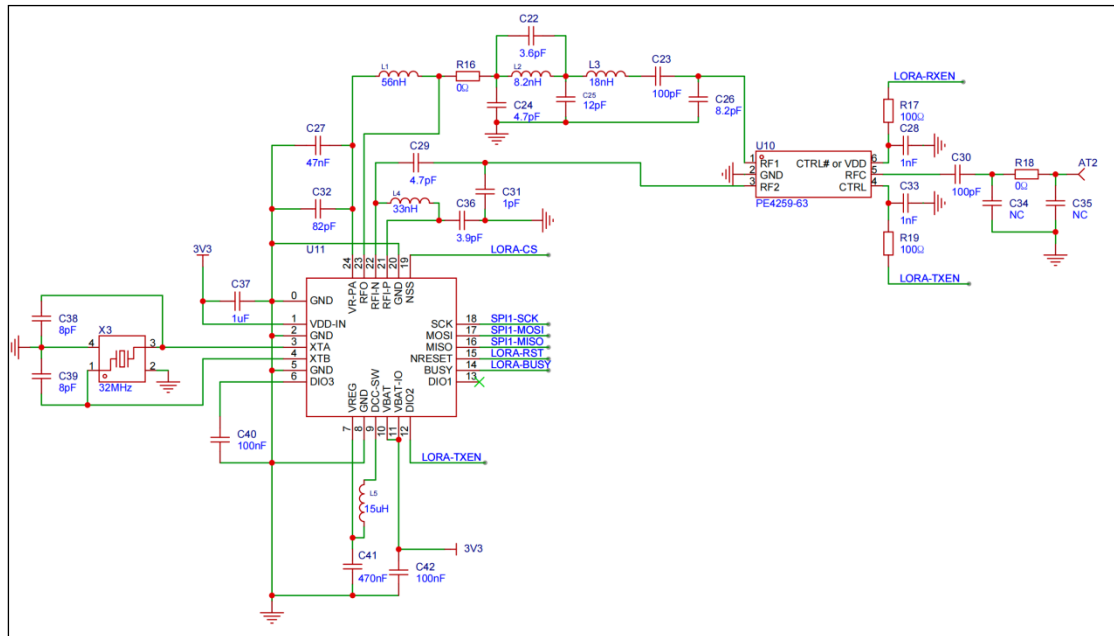
5.2.1.4 GPS 模块硬件 CubeMX 设置



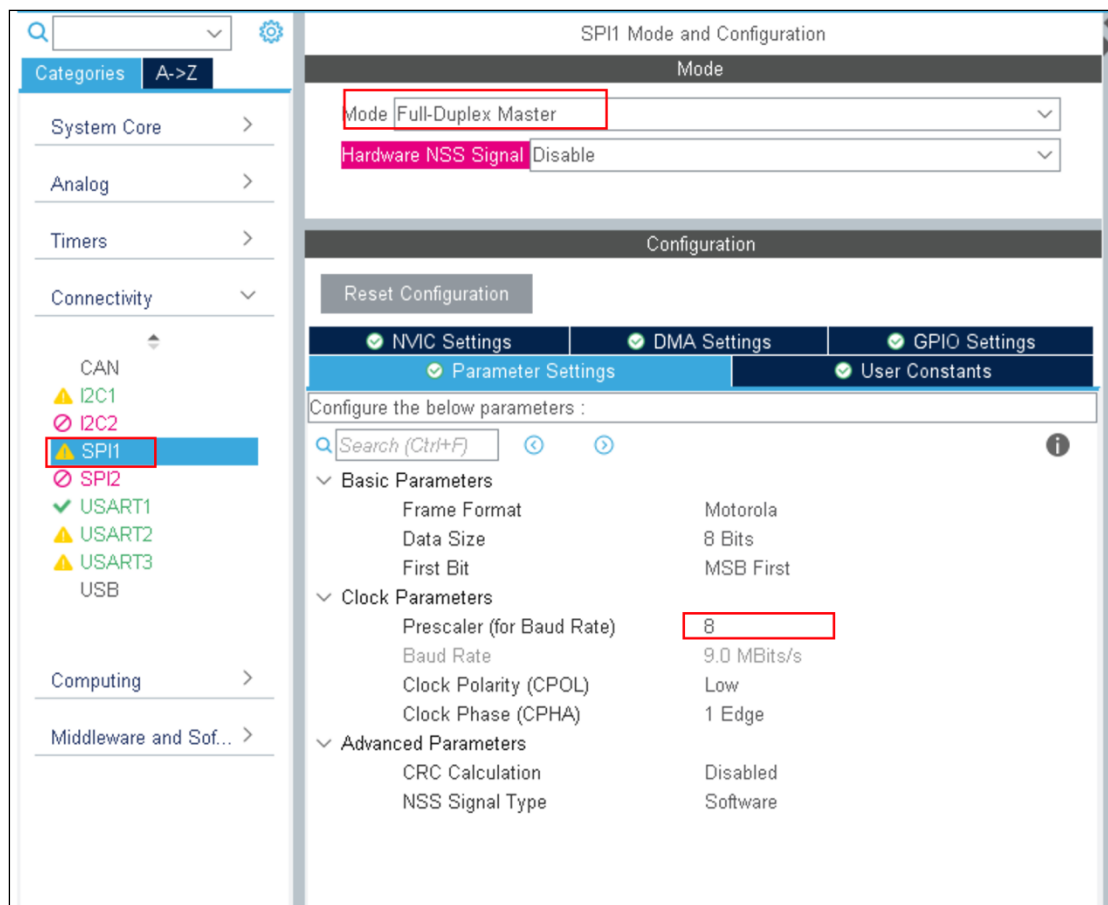
Parameter Settings			
NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
USART2 global interrupt	☒	0	0

GPIO							
Search Signals							
Search (Ctrl+F)							
Pin Name	Signal on Pin	GPIO outp...	GPIO mode	GPIO Pull...	Maximum ...	User Label	Modified
PA4	n/a	High	Output Pu...	No pull-up ...	Low	LoRa_CS	☒
PB0	n/a	Low	Output Pu...	No pull-up ...	Low	LoRa_RST	☒
PB1	n/a	n/a	Input mode	No pull-up ...	n/a	LoRa_BUSY	☒
PB2	n/a	High	Output Pu...	No pull-up ...	Low	LoRa_TxEN	☒
PB3	n/a	High	Output Pu...	No pull-up ...	Low	GPS_EN	☒
PB5	n/a	Low	Output Pu...	No pull-up ...	Low	DS3553_CS	☒
PB12	n/a	High	Output Pu...	No pull-up ...	Low	LoRa_RxEN	☒
PB13	n/a	Low	Output Pu...	No pull-up ...	Low	QS100_W...	☒

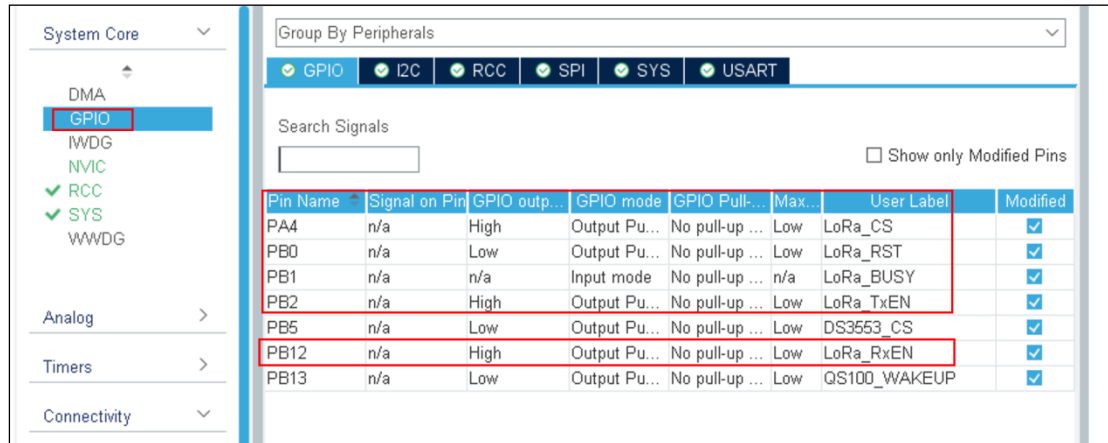
5.2.1.5 LoRa 模块硬件连接电路



5.2.1.6 LoRa 模块硬件 CubeMX 设置



The screenshot shows the SPI1 Mode and Configuration window in CubeMX. The left sidebar lists various components, with SPI1 highlighted. The main window displays the configuration for SPI1, including Mode (Full-Duplex Master), Hardware NSS Signal (Disable), and Configuration parameters. The Configuration section includes Basic Parameters (Frame Format: Motorola, Data Size: 8 Bits, First Bit: MSB First), Clock Parameters (Prescaler: 8, Baud Rate: 9.0 MBits/s, Clock Polarity: Low, Clock Phase: 1 Edge), and Advanced Parameters (CRC Calculation: Disabled, NSS Signal Type: Software).



The screenshot shows the GPIO configuration window in CubeMX. The left sidebar lists various components, with GPIO highlighted. The main window displays the GPIO configuration table, which lists pins and their configurations. The table is filtered by the 'Show only Modified Pins' checkbox.

Pin Name	Signal on Pin	GPIO outp...	GPIO mode	GPIO Pull...	Max...	User Label	Modified
PA4	n/a	High	Output Pu...	No pull-up ...	Low	LoRa_CS	✓
PB0	n/a	Low	Output Pu...	No pull-up ...	Low	LoRa_RST	✓
PB1	n/a	n/a	Input mode	No pull-up ...	n/a	LoRa_BUSY	✓
PB2	n/a	High	Output Pu...	No pull-up ...	Low	LoRa_TxEN	✓
PB5	n/a	Low	Output Pu...	No pull-up ...	Low	DS3553_CS	✓
PB12	n/a	High	Output Pu...	No pull-up ...	Low	LoRa_RxEN	✓
PB13	n/a	Low	Output Pu...	No pull-up ...	Low	QS100_WAKEUP	✓

5.2.2 驱动层

使用工具生成的代码，不需要做变动。

5.2.3 硬件接口层之 NB-IoT

5.2.3.1 Inf_QS100.h

```
#ifndef __INF_QS100_H
```

更多 Java -大数据 -前端 -python 人工智能资料下载，可百度访问：尚硅谷官网

```
#define __INF_QS100_H

#include "Debug.h"
#include "Tool.h"

/* 初始化 QS100 模块 */
void Inf_QS100_Init(void);

/* 测试模块是否能够连接外网 */
CommonStatus Inf_QS100_GetIP(void);

/* 创建本地客户端 */
CommonStatus Inf_QS100_CreateClient(uint16_t port, uint8_t *socket_id);

/* 连接远程服务端 */
CommonStatus Inf_QS100_ConnectServer(uint8_t socket_id, uint8_t *ip, uint16_t port);

/* 关闭连接 */
CommonStatus Inf_QS100_CloseSocket(uint8_t socket_id);

/* 发送数据 */
CommonStatus Inf_QS100_SendData(uint8_t *data, uint16_t len);

/* 进入低功耗 */
void Inf_QS100_EnterLowPower(void);

/* 退出低功耗 */
void Inf_QS100_ExitLowPower(void);

#endif
```

5.2.3.2 Inf_QS100.c

```
#include "Inf_QS100.h"

uint8_t QS100_buff[128] = {0};
uint8_t QS100_buff_len = 0;

uint8_t response_buff[512] = {0};
uint16_t response_buff_len = 0;

#define SEQUENCE_ID 1
#define NUM_RETRIES 5

void Inf_QS100_HandleResponse(void)
{
    // 每次读取数据清空上一次信息
    memset(response_buff, 0, 512);
    response_buff_len = 0;

    uint8_t count = NUM_RETRIES;
    do
    {
        // 接收的一次信息
        memset(QS100_buff, 0, 128);
        HAL_UART_Receive(&huart3, QS100_buff, 128, 1000);
        QS100_buff_len = strlen((char *)QS100_buff);

        // 拼接接收的信息
        memcpy(&response_buff[response_buff_len], QS100_buff, QS100_buff_len);
        response_buff_len += QS100_buff_len;

    } while ((strstr((char *)response_buff, "OK") != NULL) && count--);
    debug_printfln("%s", response_buff);
}
```

```
void Inf_QS100_SendCmd(uint8_t *cmd)
{
    /* 1. 发送命令 */
    HAL_UART_Transmit(&huart3, cmd, strlen((char *)cmd), 1000);
    /* 2. 处理回复 */
    // 接收 3 次消息 或者收到 OK
    Inf_QS100_HandleResponse();
}

void Inf_QS100_Init(void)
{
    /* 1. 初始化驱动 */
    // MX_USART3_UART_Init();

    /* 2. 重置 QS100 */
    Inf_QS100_SendCmd("AT+RB\r\n");

    /* 3. 设置串口回显 */
    Inf_QS100_SendCmd("ATE1\r\n");

    /* 4. 查询版本号 */
    Inf_QS100_SendCmd("AT+CGMR\r\n");
}

CommonStatus Inf_QS100_GetIP(void)
{
    /* 1. 查询附着 */
    Inf_QS100_SendCmd("AT+CGATT?\r\n");
    if (strstr((char *)response_buff, "+CGATT:1") == NULL)
    {
        return COMMON_ERROR;
    }
    return COMMON_OK;
}
```

```
}

CommonStatus Inf_QS100_CreateClient(uint16_t port, uint8_t *socket_id)
{
    uint8_t cmd[128] = {0};
    sprintf((char *)cmd, "AT+NSOCR=STREAM,6,%d,0\r\n", port);
    Inf_QS100_SendCmd(cmd);

    if (strstr((char *)response_buff, "OK") != NULL)
    {
        // debug_printfln("%s", response_buff);
        sscanf((char *)response_buff, "%*[^:]:%hhu", socket_id);
        debug_printfln("socket_id:%hhu", *socket_id);
        return COMMON_OK;
    }
    return COMMON_ERROR;
}

CommonStatus Inf_QS100_ConnectServer(uint8_t socket_id, uint8_t *ip, uint16_t port)
{
    uint8_t cmd[128] = {0};
    // 拼接命令
    sprintf((char *)cmd, "AT+NSOCO=%d,%s,%d\r\n", socket_id, ip, port);
    // 发送命令
    Inf_QS100_SendCmd(cmd);
    // 确认是否连接成功
    if (strstr((char *)response_buff, "OK") == NULL)
    {
        return COMMON_ERROR;
    }
    return COMMON_OK;
}
```

```
CommonStatus Inf_QS100_CloseSocket(uint8_t socket_id)
{
    uint8_t cmd[128] = {0};
    sprintf((char *)cmd, "AT+NSOCL=%d\r\n", socket_id);
    Inf_QS100_SendCmd(cmd);

    // debug_printf\n
    if (strstr((char *)response_buff, "OK") == NULL)
    {
        return COMMON_ERROR;
    }
    return COMMON_OK;
}

CommonStatus Inf_QS100_SendDataToTCP(uint8_t socket_id, uint8_t sequenceId, uint8_t
*data, uint16_t len)
{
    // 修改数据为 16 位进制数字
    uint8_t hex_data[len * 2 + 1];
    memset(hex_data, 0, sizeof(hex_data));

    for (uint16_t i = 0; i < len; i++)
    {
        sprintf((char *)&hex_data[i * 2], "%02x", data[i]);
    }

    uint8_t cmd[128] = {0};
    // 拼接命令
    sprintf((char *)cmd, "AT+NSOSD=%d,%d,%s,0x200,%d\r\n", socket_id, len,
hex_data, sequenceId);
    // 发送命令
    Inf_QS100_SendCmd(cmd);
    if (strstr((char *)response_buff, "OK") == NULL)
```

```
{
    return COMMON_ERROR;
}

// 查询发送状态
memset(cmd, 0, 128);
sprintf((char *)cmd, "AT+SEQUENCE=%d,%d\r\n", socket_id, sequenceId);
do
{
    Inf_QS100_SendCmd(cmd);

    // debug_printfln("=====");
    // for (uint16_t i = 0; i < response_buff_len; i++)
    // {
    //     debug_printfln("%d:%c", i, response_buff[i]);
    // }

    // debug_printfln("=====");
} while (response_buff[response_buff_len - 9] == '2');
if (response_buff[response_buff_len - 9] == '0')
{
    return COMMON_ERROR;
}

return COMMON_OK;
}

CommonStatus Inf_QS100_SendData(uint8_t *data, uint16_t len)
{
    uint8_t count = NUM_RETRIES;

    /* 1. 测试外网连接 */
    while (count-- && Inf_QS100_GetIP() != COMMON_OK)
    {
        Delay_ms(2);
    }
}
```

```
if (count == 0)
{
    debug_printfln("Inf_QS100_GetIP error");
    return COMMON_ERROR;
}

/* 2. 创建本地客户端 */
count = NUM_RETRIES;
uint8_t socket_id = 0xff;
while (count-- && Inf_QS100_CreateClient(8080, &socket_id) != COMMON_OK)
{
    Delay_ms(2);
}
if (count == 0)
{
    debug_printfln("Inf_QS100_CreateClient error");
    return COMMON_ERROR;
}

/* 3. 连接远程服务端 */
count = NUM_RETRIES;
while (count-- && Inf_QS100_ConnectServer(socket_id, TCP_SERVER_IP,
TCP_SERVER_PORT) != COMMON_OK)
{
    Delay_ms(2);
}
if (count == 0)
{
    debug_printfln("Inf_QS100_ConnectServer error");
    return COMMON_ERROR;
}

/* 4. 发送数据 */
```

```
count = NUM_RETRIES;

while (count-- && Inf_QS100_SendDataToTCP(socket_id, SEQUENCE_ID, data, len) !=
COMMON_OK)
{
    Delay_ms(2);
}

if (count == 0)
{
    debug_printfln("Inf_QS100_SendDataToTCP error");
    return COMMON_ERROR;
}

/* 5. 断开连接 */
count = NUM_RETRIES;

while (count-- && Inf_QS100_CloseSocket(socket_id) != COMMON_OK)
{
    Delay_ms(2);
}

if (count == 0)
{
    debug_printfln("Inf_QS100_CloseSocket error");
    return COMMON_ERROR;
}

return COMMON_OK;
}

void Inf_QS100_EnterLowPower(void)
{
    /* 1. 直接发送进入低功耗的命令 */
    debug_printfln("准备进入低功耗模式");
    Inf_QS100_SendCmd("AT+FASTOFF=0\r\n");
}

void Inf_QS100_ExitLowPower(void)
{

```

```
debug_printfln("退出低功耗模式");
HAL_GPIO_WritePin(NB_WKUP_GPIO_Port, NB_WKUP_Pin, GPIO_PIN_SET);
Delay_s(2);
HAL_GPIO_WritePin(NB_WKUP_GPIO_Port, NB_WKUP_Pin, GPIO_PIN_RESET);
}
```

5.2.4 硬件接口层之 GPS

5.2.4.1 Inf_AT6558R.h

```
#ifndef __INF_AT6558R_H
#define __INF_AT6558R_H

#include "usart.h"
#include "stdio.h"
#include "string.h"

void Inf_AT6558R_Init(void);
void Inf_AT6558R_ReadData(uint8_t data[], uint16_t *dataLen);
void Inf_AT6558R_EnterLowPower(void);
void Inf_AT6558R_ExitLowPower(void);
#endif
```

5.2.4.2 Inf_AT6558R.c

```
#include "Inf_AT6558R.h"

#define GPS_MODE "PCAS04,3"
#define GPS_UPDAE_FREQ "PCAS02,1000"
#define GPS_INFO_BUFF_SIZE 1024
#define GPS_INFO_BEGIN "GGA"
#define GPS_INFO_END "TXT"

/* 接收 gps 信息的缓冲区数组要足够大, 否则信息接收不完整, 容易丢失数据 1k 足够 */
static uint8_t rxBuff[GPS_INFO_BUFF_SIZE] = {0}; /* 产生一次中断接收的数据 */
```

更多 Java - 大数据 - 前端 - python 人工智能资料下载, 可百度访问: 尚硅谷官网

```
static uint16_t rxSize = 0; /* 产生一次中断接收到的数据的长度 */

/*
当碰到空闲帧 或者收到的数据大于等于 256
size: 表示实际收到的字节数
*/
void Inf_AT6558R_UartCallback(uint16_t size)
{
    /* 重新调用, 给下次接收做准备 */
    HAL_UARTEx_ReceiveToIdle_IT(&huart2, rxBuff, GPS_INFO_BUFF_SIZE);
    rxSize = size;
}

static uint8_t tmpCmd[50] = {0};
/*
$PCAS02,fixInt*CS<CR><LF>
命令格式:
    $PCAS02 消息 ID, 语句头
    fixInt 定位更新时间间隔, 单位为 ms。
    CS 校验和, $和*之间 (不包括$和*) 所有字符的异或结果 16 进制

传的参数: $和*中间的部分
*/
void Inf_AT6558R_SendCmd(uint8_t *cmd)
{
    uint16_t tmp = cmd[0];
    for(uint8_t i = 1; cmd[i] != '\0'; i++)
    {
        tmp ^= cmd[i];
    }

    sprintf((char *)tmpCmd, "$%s*%X\r\n", cmd, tmp);
    HAL_UART_Transmit(&huart2, tmpCmd, strlen((char *)tmpCmd), 2000);
}
```

```
    debug_printfln("tmpCmd = %s", tmpCmd);
}

void Inf_AT6558R_Init(void)
{
    MX_USART2_UART_Init(); /* 初始化串口 2 */

    /* 使用中断的方式接收数据 */
    HAL_UARTEx_ReceiveToIdle_IT(&huart2, rxBuff, GPS_INFO_BUFF_SIZE);

    /* 配置 gps+北斗模式 */
    debug_printfln("定位模块配置为 GPS+北斗模式");
    Inf_AT6558R_SendCmd(GPS_MODE);

    /* 定位更新频率 1HZ */
    debug_printfln("定位模块配置 定位更新频率为 1Hz");
    Inf_AT6558R_SendCmd(GPS_UPDAE_FREQ);
}

/**
 * @description: 读取 gps 定位信息 一次完整的定位信息是由
 * 当使用 gps+bd 定位时:
 * $GNGGA,042236.000,,,,,0,00,25.5,,,,, *7B
 * $GNGLL,,,,,042236.000,V,N*65
 * $GNGSA,A,1,,,,,,,,,25.5,25.5,25.5,1*01
 * $GNGSA,A,1,,,,,,,,,25.5,25.5,25.5,4*04
 * $GPGSV,1,1,00,0*65
 * $BDGSV,1,1,02,13,,25,46,,28,0*7B
 * $GNRMC,042236.000,V,,,,,020624,,,N,V*2A
 * $GNVTG,,,,,,,,N*2E
 * $GNZDA,042236.000,02,06,2024,00,00*49
 * $GPTXT,01,01,01,ANTENNA OPEN*25

```

```
    总是以：$GNGGA 开始
        $GPTXT 结束
*
* @param {uint8_t} data 读取到的数据
* @param {uint16_t} *dataLen 读取到的数据长度
*/
void Inf_AT6558R_ReadData(uint8_t data[], uint16_t *dataLen)
{
    /* 清空缓冲区数组 */
    *dataLen = 0;
    memset(data, 0, strlen((char *)data));

    /* 如果没有收到数据，直接返回 */
    if(rxSize == 0) return;

    /* 收到的数据中：既包括信息的开头也包括信息的结束 */
    memcpy(&data[0], rxBuff, rxSize);
    if(strstr((char *)data, GPS_INFO_BEGIN) && strstr((char *)data, GPS_INFO_END))
    {
        *dataLen = rxSize;
    }
    rxSize = 0;
}

/**
 * @description: 使 gps 模块进入低功耗模式
 * 模块本身没有提供低功耗模式,我们通过断开模块的电源达到省电的目的
 * @return {*}
 */
void Inf_AT6558R_EnterLowPower(void)
{
    debug_printfln("GPS 模块电源关闭...");
    HAL_GPIO_WritePin(GPS_EN_GPIO_Port, GPS_EN_Pin, GPIO_PIN_RESET);
}
```

```

}

/**
 * @description: 使 gps 模块退出低功耗模式
 * @return {*}
 */
void Inf_AT6558R_ExitLowPower(void)
{
    debug_printfln("GPS 模块电源打开...");
    HAL_GPIO_WritePin(GPS_EN_GPIO_Port, GPS_EN_Pin, GPIO_PIN_SET);
}

```

5.2.5 硬件接口层之 LoRa

5.2.5.1 移植 LoRa 模块官方驱动

具体移植方法参考下面的文档



LoRa驱动移植.doc

x

5.2.5.2 Inf_LoRa.h

```

#ifndef __INF_LORA_H
#define __INF_LORA_H

#include "ebyte_core.h"
#include "main.h"

void Inf_LoRa_Init(void);

void Inf_LoRa_SendData(uint8_t data[], uint8_t len);

void Inf_LoRa_ReadData(void);

#endif

```

5.2.5.3 Inf_LoRa.c

```
#include "Inf_LoRa.h"
#include "stdio.h"
#include "spi.h"

/* 调用一次 */
void Inf_LoRa_Init(void)
{
    // Ebyte_E220x_Init();
    MX_SPI1_Init();

    Ebyte_RF.Init();
}

void Inf_LoRa_SendData(uint8_t data[], uint8_t len)
{
    Ebyte_RF.Send(data, len, 0);
}

/* 要在循环中调用 */
void Inf_LoRa_ReadData(void)
{
    Ebyte_RF.StartPollTask();
}

void LoRa_ReceiveSuccessCallback(uint8e_t *buffer, uint8e_t length)
{
    printf("收到数据: 长度=%d, 内容=.*s\r\n", length, length, buffer);
}

void LoRa_TransmitSuccessCallback(void)
{
    printf("发送成功....\r\n");
}
```

```
}
```

5.2.6 应用层

5.2.6.1 App_Communication.h

```
#ifndef __APP_COMMUNICATION_H
#define __APP_COMMUNICATION_H

#include "Common_Config.h"

#include "Inf_AT6558R.h" /* GPS */
#include "Inf_QS100.h" /* NB-IoT */
#include "Inf_LoRa.h" /* LoRa */

void App_Cummunication_Start(void);
void App_Cummunication_SendData(UploadDataType *uploadData);
CommonStatus App_Cummunication_SendDataByNBIoT(UploadDataType *uploadData);
void App_Cummunication_SendDataByLoRa(UploadDataType *uploadData);
void App_Cummunication_GetGpsInfo(UploadDataType *uploadData);
void App_Cummunication_FormatDataToJson(UploadDataType *uploadData);
void printUploadData(const UploadDataType *data);
#endif
```

5.2.6.2 App_Communication.c

```
#include "App_Communication.h"
#include "cJSON.h"

/**
 * @description: 通讯模块启动
 * @return {*}
 */
void App_Cummunication_Start(void)
{
    Inf_QS100_Init();
    Inf_AT6558R_Init();
    Inf_LoRa_Init();
}
```

更多 [Java](#) -[大数据](#) -[前端](#) -[python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
}

/**
 * @description: 发送数据
 * @param {uint8_t} data 要发送的数据
 * @param {uint16_t} dataLen 要发送的数据长度
 * @return {*}
 */
void App_Cummuication_SendData(UploadDataType *uploadData)
{
    /* 1. 数据格式为 Json 格式发送 */
    App_Cummuication_FormatDataToJson(uploadData);
    /* 2. 先通过 IoT 向外发送 */
    CommonStatus sendStatus = App_Cummuication_SendDataByNBioT(uploadData);
    /* 3. 如果 IoT 发送失败,再通过 LoRa 向外发送 */
    if(sendStatus == COMMON_ERROR)
    {
        App_Cummuication_SendDataByLoRa(uploadData);
    }
}

/**
 * @description: 通过 NBioT 发送数据
 * @param {uint8_t} gspData gps 数据
 * @param {uint8_t} gpsDataLen gps 数据的长度
 * @param {uint16_t} stepNum 步数
 * @return {*}
 */
CommonStatus App_Cummuication_SendDataByNBioT(UploadDataType *uploadData)
{
    return Inf_QS100_SendTcpData(TCP_SERVER_IP, TCP_SERVER_PORT, uploadData->data,
uploadData->dataLen);
    // return Inf_QS100_SendTcpData(TCP_SERVER_IP, TCP_SERVER_PORT, "abc", 3);
}
```

```
}

/**
 * @description: 通过 LoRa 发送数据
 * @param {uint8_t} gspData gps 数据
 * @param {uint8_t} gpsDataLen gps 数据的长度
 * @param {uint16_t} stepNum 步数
 * @return {*}
 */
void App_Communication_SendDataByLoRa(UploadDataType *uploadData)
{
    Inf_LoRa_SendData(uploadData->data, (uint8_t)uploadData->dataLen);
}

static uint8_t  gpsInfoBuff[1024];
static uint16_t gspsInfoLen;

/**
 * @description: 获取 gps 信息
 * $GNGGA,070822.000,4006.81888,N,11621.89413,E,1,05,25.5,30.2,M,-9.6,M,,*58
 * $GNGLL,4006.81888,N,11621.89413,E,070822.000,A,A*49
 * $GNGSA,A,3,05,12,25,,,,,,,,,25.5,25.5,25.5,1*02
 * $GNGSA,A,3,10,13,,,,,,,,,25.5,25.5,25.5,4*05
 * $GPGSV,3,1,11,05,54,263,24,06,32,098,,09,20,043,,11,68,061,,0*61
 * $GPGSV,3,2,11,12,14,237,17,13,23,176,,15,05,204,,19,08,152,,0*60
 * $GPGSV,3,3,11,20,75,327,,25,13,265,23,29,28,314,,0*51
 * $BDGSV,2,1,05,08,64,301,,10,47,216,15,13,56,283,10,27,59,133,,0*78
 * $BDGSV,2,2,05,38,72,326,,0*48
 * $GNRMC,070822.000,A,4006.81888,N,11621.89413,E,0.81,359.02,020624,,A,V*02
 * $GNVTG,359.02,T,,M,0.81,N,1.51,K,A*22
 * $GNZDA,070822.000,02,06,2024,00,00*47
 * $GPTXT,01,01,01,ANTENNA OPEN*25
 * @param {uint8_t} gpsInfo gps 信息
 * @param {uint16_t} *gpsInfoLen gps 信息的长度
```

```
* @return {*}
*/
void App_Communication_GetGpsInfo(UploadDataType *uploadData)
{
    debug_printfln("读取 Gps 定位信息");
    char *gnrmcStart;
    /* 得到有效的 gps 数据 */
    while(1)
    {
        Inf_AT6558R_ReadData(gpsInfoBuff, &gspsInfoLen);

        if(gspsInfoLen > 0)
        {
            uint8_t valid = 0;
            gnrmcStart = strstr((char *)gpsInfoBuff, "$GNRMC");
            // gnrmcStart =
            "$GNRMC,220822.000,A,4006.81888,N,11621.89413,E,0.81,359.02,020624,,A,V*02";
            sscanf(gnrmcStart, "%*[^AV]%c", &valid);
            if(valid == 'A')
            {
                debug_printfln("有效定位数据:\r\n%s", gpsInfoBuff);
                break;
            }
            debug_printfln("无效定位数据:\r\n%s", gpsInfoBuff);
        }
    }

    /* 解析 gps 信息 */
    char time[7]; /* 时分秒 HHmmss */
    char date[7]; /* 日月年 ddmmyy*/

    sscanf(gnrmcStart,
            "$GNRMC,%6c%*7c%lf,%c,%lf,%c,%lf,%*f,%6c",
```

```
        time,
        &uploadData->lat,
        (char *)&uploadData->latDir,
        &uploadData->lon,
        (char *)&uploadData->lonDir,
        &uploadData->speed,
        date);

/* 把那时间格式为: yyyy-MM-dd HH:mm:ss */
sprintf((char *)uploadData->dateTime,
        "20%c%c-%c%c-%c%c %c%c:%c%c:%c%c",
        date[4],
        date[5],
        date[2],
        date[3],
        date[0],
        date[1],
        time[0],
        time[1],
        time[2],
        time[3],
        time[4],
        time[5]);

/* 再把 utc 时间换算成 东八区时间(北京时间) */
Common_Tool_UtcTime2BeijTime(uploadData->dateTime, uploadData->dateTime);

/* 经纬度信息调整:
    纬度: 4006.81888 其中 40 表示° 06.81888 表示分
    经度: 11621.89413 其中 116 表示° 21.89413 表示分

    我们需要把经纬度信息调整为 xxx.xx° 方便后续处理
*/
```

```
        uploadData->lat = (uint8_t)(uploadData->lat / 100) + (uploadData->lat -
(uint8_t)(uploadData->lat / 100) * 100) / 60;

        uploadData->lon = (uint8_t)(uploadData->lon / 100) + (uploadData->lon -
(uint8_t)(uploadData->lon / 100) * 100) / 60;
    }

/**
 * @description: 对 gps 信息和计步信息进行处理
 * 把 gps 信息和步数信息,格式化为 json 格式数据
 */
void App_Communication_FormatDataToJson(UploadDataType *uploadData)
{
    /* 补充 id 信息,方便识别设备信息*/
    sprintf((char *)uploadData->uuid, "%08X%08X%08X", HAL_GetUIDw2(),
HAL_GetUIDw1(), HAL_GetUIDw0());

    /* 先创建 json 对象 */
    cJSON *json = cJSON_CreateObject();

    /* 向对象中添加键值对 */
    cJSON_AddStringToObject(json, "uuid", (char *)uploadData->uuid);

    cJSON_AddNumberToObject(json, "lon", uploadData->lon);
    cJSON_AddStringToObject(json, "lonDir", (char *)uploadData->lonDir);

    cJSON_AddNumberToObject(json, "lat", uploadData->lat);
    cJSON_AddStringToObject(json, "latDir", (char *)uploadData->latDir);

    cJSON_AddNumberToObject(json, "speed", uploadData->speed);
    cJSON_AddStringToObject(json, "dateTime", (char *)uploadData->dateTime);
    cJSON_AddNumberToObject(json, "stepNum", uploadData->stepNum);

    char *out = cJSON_PrintUnformatted(json);
```

```
memcpy(uploadData->data, out, strlen(out));
uploadData->dataLen = strlen(out);

cJSON_Delete(json); /* 释放对象 防止内存溢出 */

printUploadData(uploadData);
}

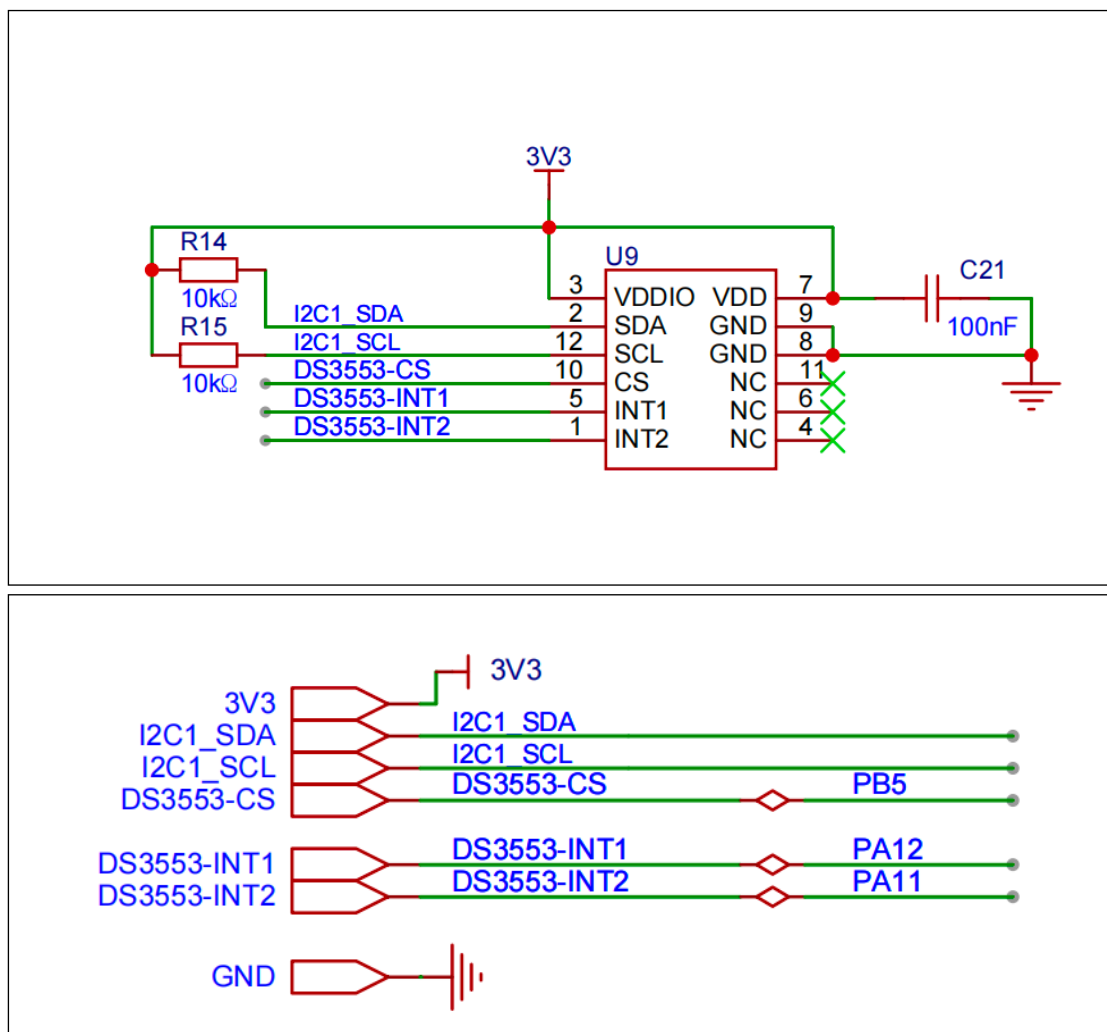
void printUploadData(const UploadDataType *data)
{
    printf("GPS 信息:\n");
    printf("  经度: %f, 方向: %s\n", data->lon, data->lonDir);
    printf("  纬度: %f, 方向: %s\n", data->lat, data->latDir);
    printf("  对地速度: %f 节\n", data->speed);
    printf("  定位时间: %s\n", data->dateTime);

    printf("计步信息:\n");
    printf("  运动步数: %u\n", data->stepNum);

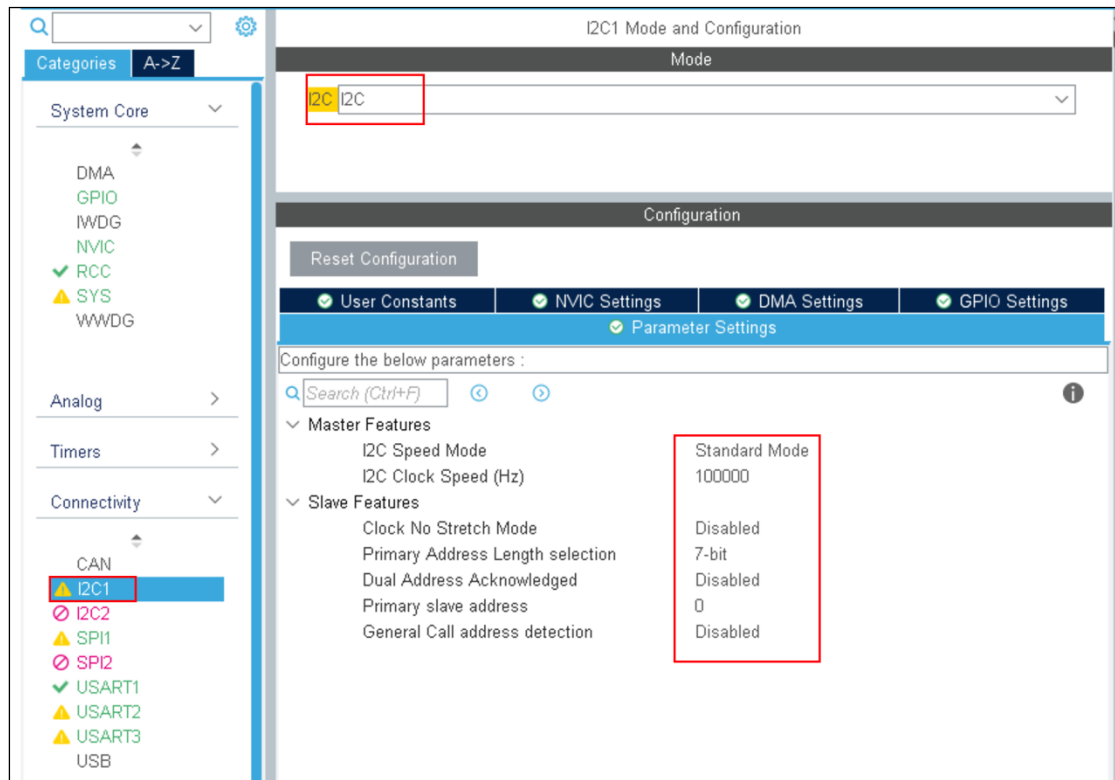
    printf("上传的数据:\n");
    printf("  数据长度: %u\n", data->dataLen);
    printf("  数据: %.*s\n", data->dataLen, data->data);
}
```

5.3 计步模块

5.3.1 硬件原理图



5.3.2 CubeMx 配置



5.3.3 驱动层

工具生成不用修改

5.3.4 硬件接口层

5.3.4.1 Inf_DS3553.h

```
#ifndef __INF_DS3553_H
#define __INF_DS3553_H

#include "i2c.h"
#include "gpio.h"
#include "Common_Debug.h"

#define DS3553_ADDR_W (0X4E)
#define DS3553_ADDR_R (0X4F)

#define DS3553_CS_H HAL_GPIO_WritePin(DS3553_CS_GPIO_Port, DS3553_CS_Pin,
GPIO_PIN_SET)
#define DS3553_CS_L HAL_GPIO_WritePin(DS3553_CS_GPIO_Port, DS3553_CS_Pin,
GPIO_PIN_RESET)
```

更多 [Java](#) -[大数据](#) -[前端](#) -[python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
#define DS3553_CHIP_ID_REG 0X01
#define DS3553_USER_SET_REG 0XC3
#define DS3553_STEP_CNT_L_REG 0XC4
#define DS3553_STEP_CNT_M_REG 0XC5
#define DS3553_STEP_CNT_H_REG 0XC6
void Inf_DS3553_Init(void);
uint8_t Inf_DS3553_ReadId(void);
uint32_t Inf_DS3553_ReadCNT(void);
#endif
```

5.3.4.2 Inf_DS3553.c

```
#include "Inf_DS3553.h"

void Inf_DS3553_Init(void)
{
    /* I2C1 初始化 */
    MX_I2C1_Init();
}

uint8_t Inf_DS3553_ReadReg(uint8_t regAddr)
{
    uint8_t result = 0;
    DS3553_CS_L;
    HAL_Delay(4);
    // HAL_I2C_Mem_Read(&hi2c1, DS3553_ADDR_R, regAddr, I2C_MEMADD_SIZE_8BIT,
    &result, 1, 2000);
    HAL_I2C_Master_Transmit(&hi2c1, DS3553_ADDR_W, &regAddr, 1, 2000);
    HAL_I2C_Master_Receive(&hi2c1, DS3553_ADDR_R, &result, 1, 2000);
    DS3553_CS_H;
    HAL_Delay(11);
    return result;
}
```

```
uint8_t Inf_DS3553_ReadId(void)
{
    return Inf_DS3553_ReadReg(DS3553_CHIP_ID_REG);
}

uint32_t Inf_DS3553_ReadCNT(void)
{
    uint8_t buff[3] = {0};

    buff[0] = Inf_DS3553_ReadReg(DS3553_STEP_CNT_L_REG);
    buff[1] = Inf_DS3553_ReadReg(DS3553_STEP_CNT_M_REG);
    buff[2] = Inf_DS3553_ReadReg(DS3553_STEP_CNT_H_REG);

    return buff[0] + (buff[1] << 8) + (buff[2] << 16);
}
```

5.3.5 应用层

5.3.5.1 App_StepCount.h

```
#ifndef __APP_STEP_COUNT_H
#define __APP_STEP_COUNT_H

#include "Common_Config.h"

#include "Inf_DS3553.h"

void App_StepCount_Start(void);

void App_StepCount_GetStepNum(UploadDataType *uploadData);

#endif
```

5.3.5.2 App_StepCount.c

```
#include "App_StepCount.h"
```

```
/**
```

更多 [Java](#) -[大数据](#) -[前端](#) -[python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
* @description: 启动计步器模块
* @return {*}
*/
void App_StepCount_Start(void)
{
    Inf_DS3553_Init();
}

/**
* @description: 获取运动的总步数
*/
void App_StepCount_GetStepNum(UploadDataType *uploadData)
{
    uploadData->stepNum = Inf_DS3553_ReadCNT();
}
```

5.4 低功耗模块

当发送成功一次数据之后,立即让整个系统进入低功耗模式(计步器除外).

STM32 直接进入待机模式, IoT 和 GPS 分别根据硬件手册选择合适的低功耗.

5.4.1 App_LowPower.h

```
#ifndef __APP_LOWPPOWER_H
#define __APP_LOWPPOWER_H

#include "rtc.h"
#include "Inf_QS100.h"
#include "Inf_AT6558R.h"

void App_LowPower_Enter(void);

void App_LowPower_Exit(void);

#endif
```

5.4.2 App_LowPower.c

更多 [Java](#) -[大数据](#) -[前端](#) -[python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
#include "App_LowPower.h"

/**
 * @description: 进入低功耗模式
 * @return {*}
 */
void App_LowPower_Enter(void)
{
    MX_RTC_Init();
    /* 1. iot 设备进入低功耗 */
    Inf_QS100_EnterLowPower();
    /* 2. gps 进入低功耗 */
    Inf_AT6558R_EnterLowPower();
    /* 3. stm32 进入低功耗 */
    /* 3.1 设置 stm32 的唤醒方式为 rtc 闹钟唤醒 */
    RTC_TimeTypeDef time;
    HAL_RTC_GetTime(&hrtc, &time, RTC_FORMAT_BIN);

    RTC_AlarmTypeDef alarm;
    alarm.AlarmTime.Hours   = time.Hours;
    alarm.AlarmTime.Minutes = time.Minutes;
    alarm.AlarmTime.Seconds = time.Seconds + 20;
    HAL_RTC_SetAlarm(&hrtc, &alarm, RTC_FORMAT_BIN);
    /* 3.2 使 stm32 进入低功耗模式 */
    debug_printfln("马上进入低功耗模式...");
    Common_Tool_DelayMS(100);
    __HAL_PWR_CLEAR_FLAG(PWR_FLAG_SB); /* 清除待机标志 */
    __HAL_PWR_CLEAR_FLAG(PWR_FLAG_WU); /* 清除唤醒标记 */
    HAL_PWR_EnterSTANDBYMode();
}

void App_LowPower_Exit(void)
{

```

```
/* 1. iot 设备正常工作模式 */
Inf_QS100_ExitLowPower();

/* 2. gps 设置正常工作模式 */
Inf_AT6558R_ExitLowPower();

}
```

5.5 main.c

```
#include "main.h"
#include "i2c.h"
#include "rtc.h"
#include "spi.h"
#include "usart.h"
#include "gpio.h"

#include "Common_Debug.h"
#include "App_Cummunication.h"
#include "App_StepCount.h"
#include "App_LowPower.h"
void SystemClock_Config(void);

UploadDataType uploadData;

int main(void)
{
    HAL_Init();

    SystemClock_Config();

    MX_GPIO_Init();

    // MX_USART1_UART_Init();
    // MX_USART3_UART_Init();
    // MX_USART2_UART_Init();
    // MX_I2C1_Init();
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
// MX_SPI1_Init();

// MX_RTC_Init();

debug_init();

debug_printfln("宠物定位器项目之定位器....");

debug_printfln("系统启动中，请稍等...");

App_LowPower_Exit(); /* 设备退出低功耗模式 */

Common_Tool_DelayMS(5000);

debug_printfln("系统启动完成!");


/* 1. 模块启动 */

/* 1.1 启动通讯模块 */

App_Cummunication_Start();

/* 1.2 启动计数模块 */

App_StepCount_Start();


/* 2. 获取需要的信息 */

/* 2.1 获取经纬度信息 */

App_Cummunication_GetGpsInfo(&uploadData);

/* 2.2 获取步数信息 */

App_StepCount_GetStepNum(&uploadData);


/* 3. 发送数据 */

App_Cummunication_SendData(&uploadData);


/* 4. 设备进入低功耗模式 */

App_LowPower_Enter();

while(1)

{

}

}
```

第 6 章 LoRa 网关

6.1 调试模块

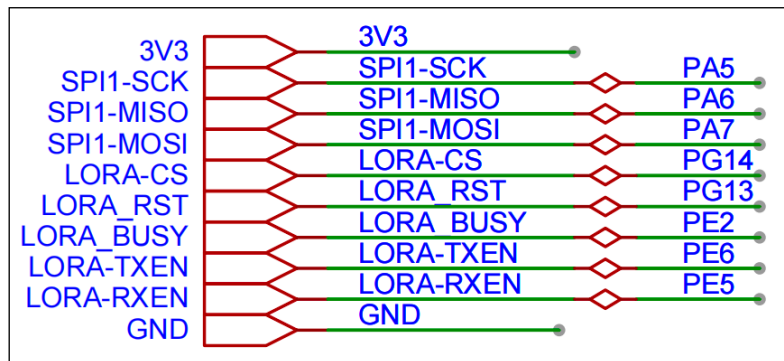
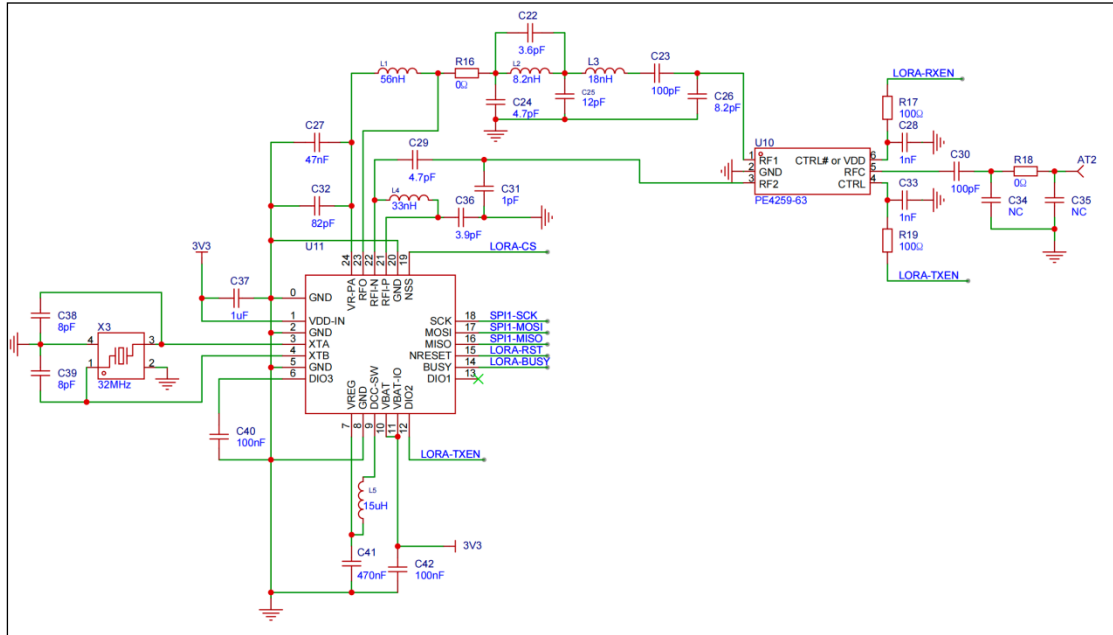
更多 [Java](#) -[大数据](#) -[前端](#) -[python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

参考定位器的调试模块设置.

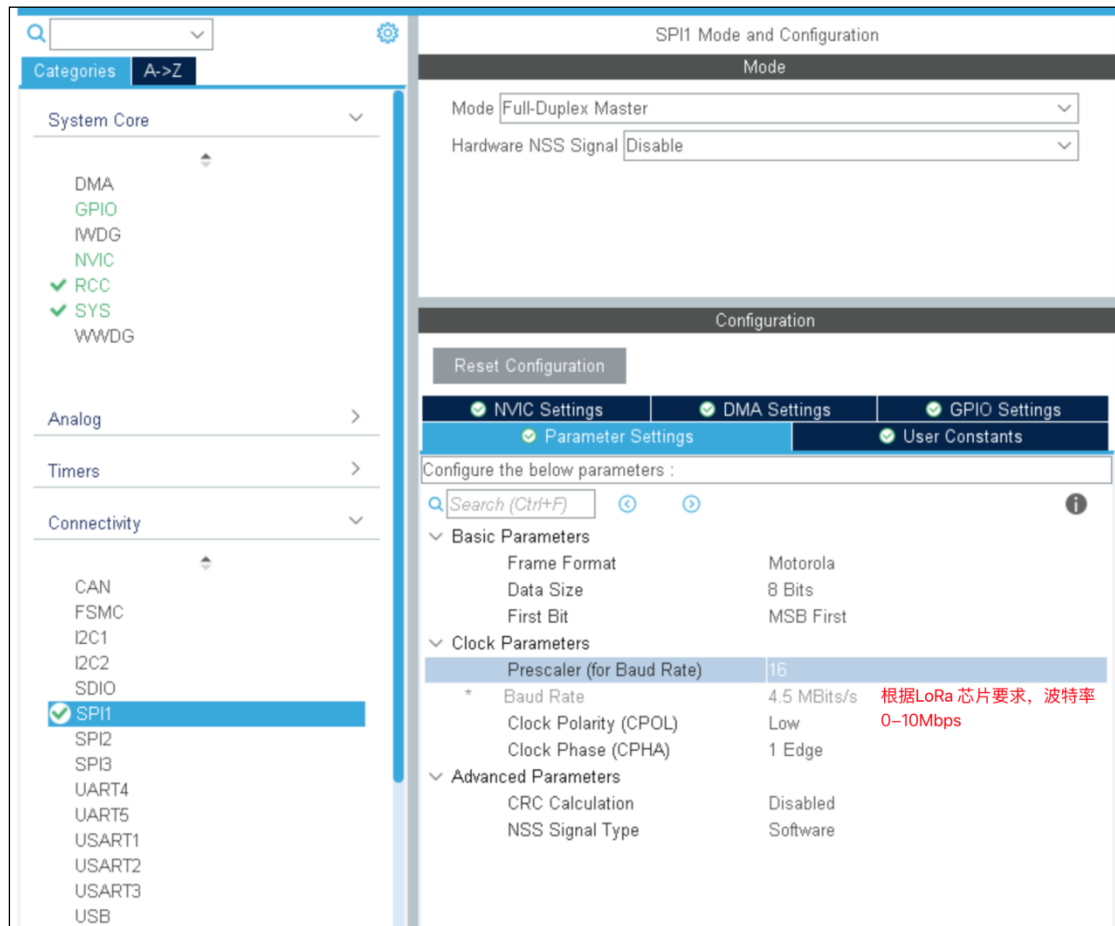
6.2 通信模块

6.2.1 CubeMx 配置

6.2.1.1 LoRa 模块硬件连接电路

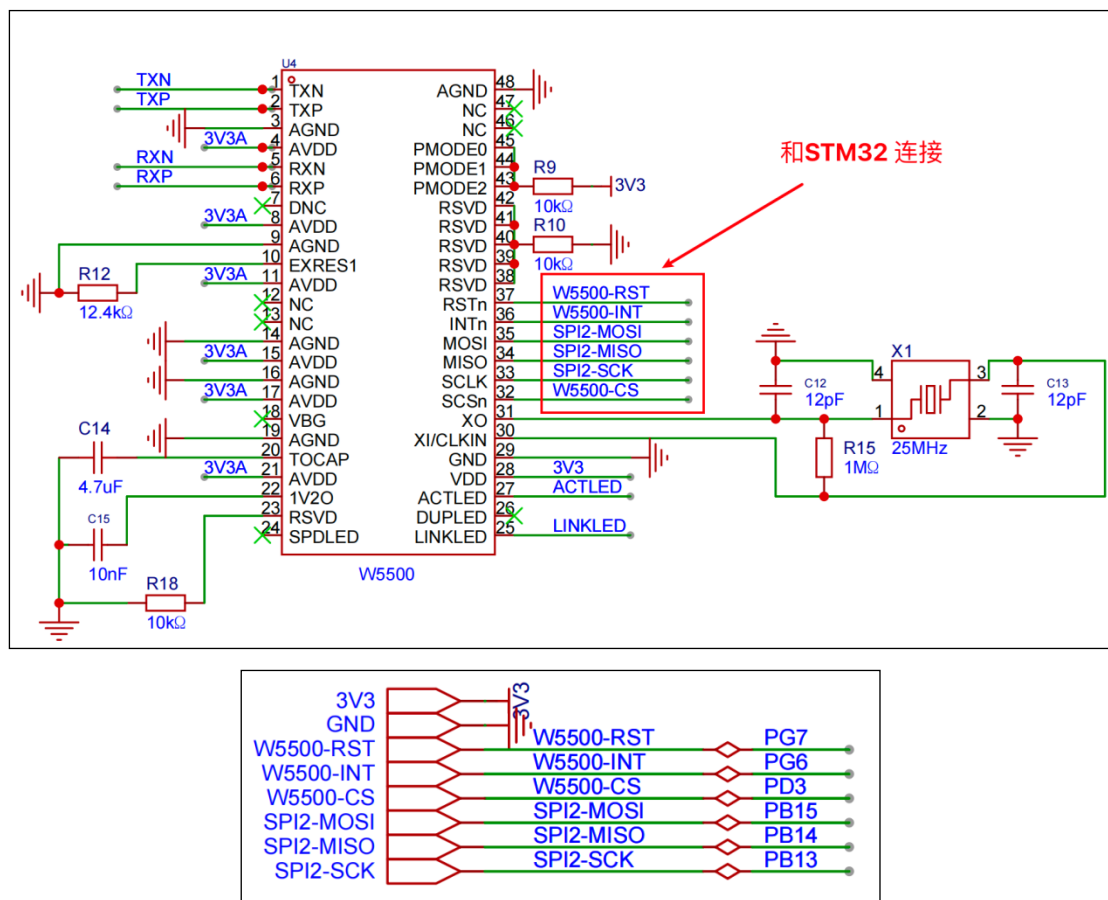


6.2.1.2 LoRa 模块硬件 CubeMx 配置

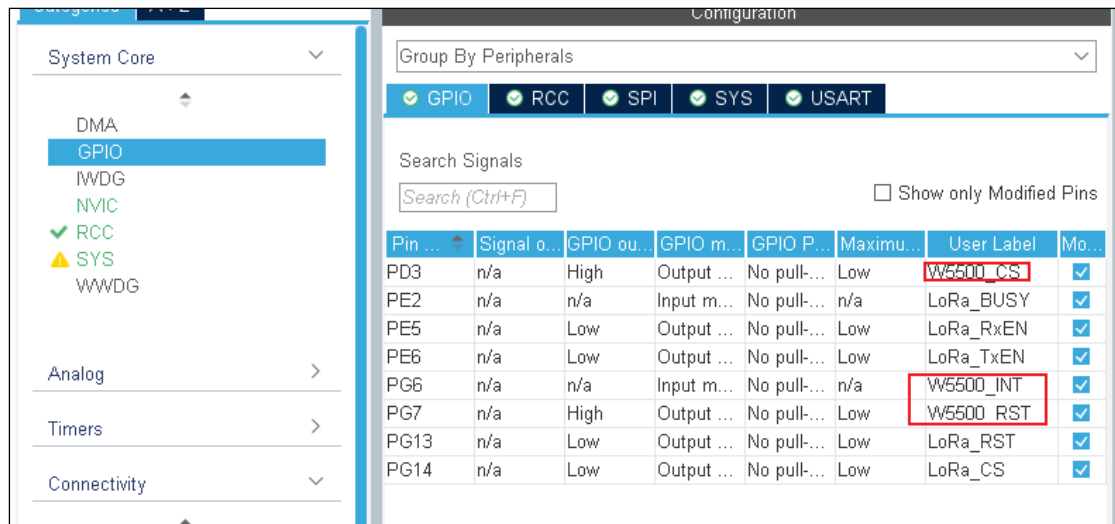
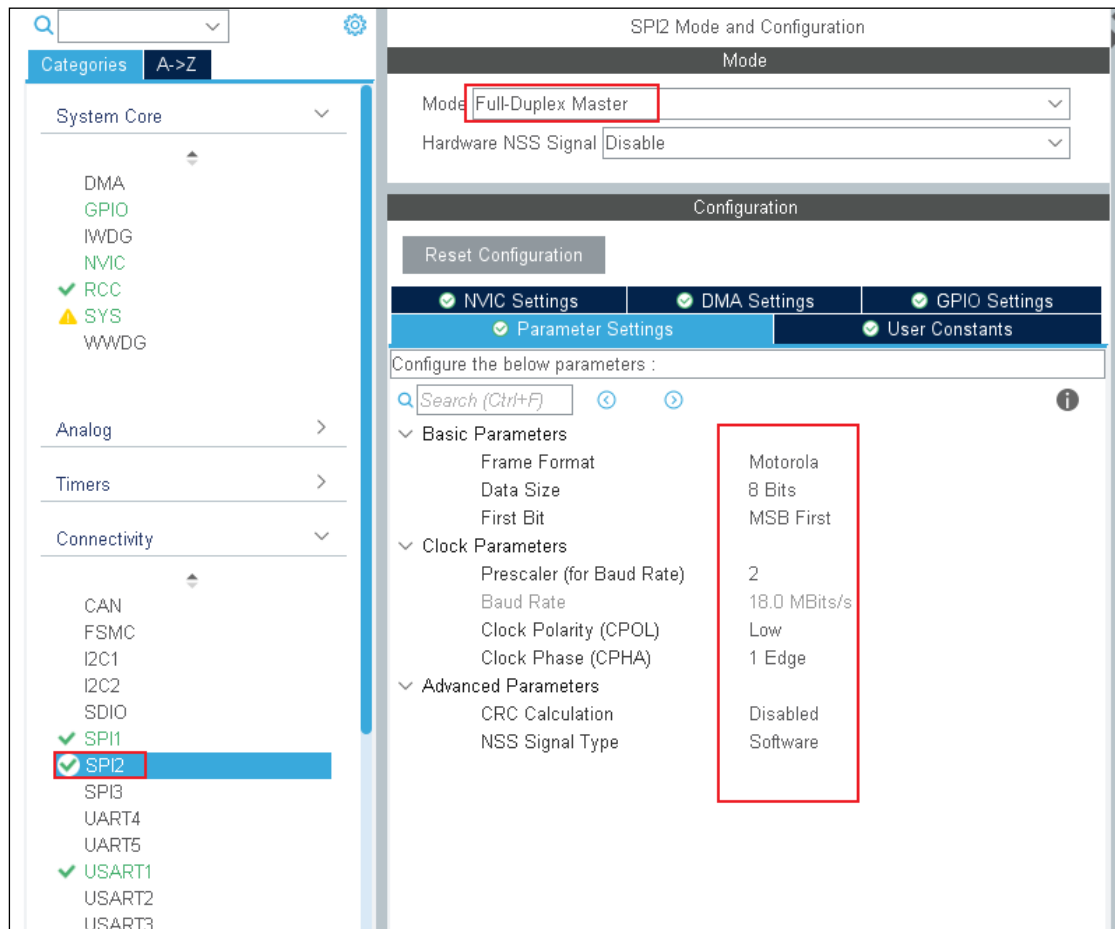


Pin N...	Signal o...	GPIO out...	GPIO m...	GPIO Pu...	Maximu...	User Label	Modified
PE2	n/a	n/a	Input mode	No pull-u...	n/a	LoRa_B...	✓
PE5	n/a	Low	Output P...	No pull-u...	Low	LoRa_Rx...	✓
PE6	n/a	Low	Output P...	No pull-u...	Low	LoRa_Tx...	✓
PG13	n/a	Low	Output P...	No pull-u...	Low	LoRa_RST	✓
PG14	n/a	Low	Output P...	No pull-u...	Low	LoRa_CS	✓

6.2.1.3 W5500 模块硬件连接电路



6.2.1.4 W5500 模块硬件 CubeMx 配置



6.2.2 驱动层

驱动层代码由工具自动生成，不需要做变动。

6.2.3 硬件接口层之 LoRa

更多 [Java](#) -[大数据](#) -[前端](#) -[python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

6.2.3.1 官方驱动移植

参考: [LoRa 驱动移植.docx](#)

6.2.3.2 Inf_LoRa.h

```
#ifndef __INF_LORA_H
#define __INF_LORA_H

#include "ebyte_core.h"

void Inf_LoRa_Init(void);
void Inf_LoRa_StartPollTask(void);
void Inf_LoRa_Send(uint8_t *buff, uint8_t size);

#endif
```

6.2.3.3 Inf_LoRa.c

```
#include "Inf_LoRa.h"

/**
 * @description: 初始化 LoRa
 */
void Inf_LoRa_Init(void)
{
    /* 1. 初始化 LoRa */
    Ebyte_RF.Init();
}

/**
 * @description: LoRa 的周期执行的函数, 建议放在主循环中
 * @return {*}
 */
void Inf_LoRa_StartPollTask(void)
{
    Ebyte_RF.StartPollTask();
}

/**
 * @description: 发送数据
 * @param {uint8_t} *buff 要发送的数据
 * @param {uint8_t} size 要发送的数据字节数
 * @return {*}
 */
void Inf_LoRa_Send(uint8_t *buff, uint8_t size)
{

```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
/* 1. 发送数据 注意超时时间必须是 0*/  
Ebyte_RF.Send(buff, size, 0);  
}
```

6.2.4 硬件接口层之 W5500

6.2.4.1 官方驱动移植

参考 STM32 高级课程

6.2.4.2 Inf_W5500.h

```
#ifndef __INF_W5500_H  
#define __INF_W5500_H  
  
#include "spi.h"  
#include "w5500.h"  
#include "socket.h"  
#include "Common_Config.h"  
#include "Common_Debug.h"  
  
void Inf_W5500_Init(void);  
  
uint8_t Inf_W5500_StartTcpClient(void);  
  
void Inf_W5500_SendTcpData(uint8_t data[], uint16_t datalen);  
  
#endif
```

6.2.4.3 Inf_W5500.c

```
#include "Inf_W5500.h"  
  
uint8_t ip[] = {192, 168, 32, 223};  
uint8_t ga[] = {192, 168, 32, 1};  
uint8_t sub[] = {255, 255, 255, 0};  
uint8_t mac[] = {110, 120, 13, 140, 150, 160};  
  
uint8_t serverIp[4] = {192, 168, 32, 228}; /* 服务端的 ip 地址: (电脑的 ip 地址) */  
uint16_t serverPort = 9999;
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
extern void user_register_function(void);

/**
 * @description: 重启 w5500 芯片
 * @return {*}
 */
static void Inf_W5500_RST(void)
{
    /* RST 引脚拉低不低于 500us */
    HAL_GPIO_WritePin(W5500_RST_GPIO_Port, W5500_RST_Pin, GPIO_PIN_RESET);
    HAL_Delay(1000);

    HAL_GPIO_WritePin(W5500_RST_GPIO_Port, W5500_RST_Pin, GPIO_PIN_SET);
    HAL_Delay(100);
}

/**
 * @description: 初始化 W5500 芯片
 * @return {*}
 */
void Inf_W5500_Init(void)
{
    /* 初始化 spi 驱动 */
    MX_SPI1_Init();

    /* 软重启芯片 */
    Inf_W5500_RST();

    /* 注册函数 */
    user_register_function();

    /* 设置 ip 地址相关 */
}
```

```
/* ip 地址 */
setSIPR(ip);
/* mac 地址 */
setSHAR(mac);
/* 子网掩码 */
setSUBR(sub);
/* 设置网关 */
setGAR(ga);

debug_printfln("ip 地址设置完成");
}

#define SN 0 /* socket 的编号 */
#define OK 1
#define FAIL 0
/* 启动一个客户端 */
uint8_t Inf_W5500_StartTcpClient(void)
{
    uint8_t status = getSn_SR(SN);

    if(status == SOCK_CLOSED) /* 表示 socket 已经关闭了 */
    {
        uint8_t r = socket(SN, Sn_MR_TCP, 8888, SF_TCP_NODELAY);
        if(r == SN)
        {
            debug_printfln("socket 0 打开成功\r\n");
        }
        else
        {
            debug_printfln("socket 0 打开失败 %d\r\n", r);
        }
    }
}
```

```
else if(status == SOCK_INIT) /* 表示 socket 的已经打开，并处于 tcp 模式 */
{
    /* 客户端需要主动去连接 tcp 服务器 */
    int8_t r = connect(SN, serverIp, serverPort);
    if(r == SOCK_OK)
    {
        debug_printfln("客户端连接服务器成功\r\n");
    }
    else
    {
        debug_printfln("客户端连接服务器失败 %d\r\n", r);
    }
}

else if(status == SOCK_CLOSE_WAIT)
{
    debug_printfln("失去与服务端的连接\r\n");
    close(SN);
}

else if(status == SOCK_ESTABLISHED) /* 如果已经建立连接 */
{
    return OK;
}

return FAIL;
}

/**
 * @description: 通过 tcp 连接发送数据
 * @param {uint8_t} data 要发送的数据
 * @param {uint16_t} dataLen 要发送的数据长度
 * @return {*}
 */
void Inf_W5500_SendTcpData(uint8_t data[], uint16_t dataLen)
```

```
{  
  
    if(dataLen == 0) return;  
  
    while(Inf_W5500_StartTcpClient() == FAIL)  
    {  
        HAL_Delay(100);  
    }  
  
    uint8_t status = getSn_SR(SN);  
    if(status == SOCK_ESTABLISHED)  
    {  
        send(SN, data, dataLen);  
    }  
}
```

6.2.5 应用层

6.2.5.1 App_Communication.h

```
#ifndef __APP_COMMUNICATION_H  
#define __APP_COMMUNICATION_H  
  
#include "Inf_LoRa.h"  
#include "Common_Debug.h"  
#include "Inf_W5500.h"  
  
void App_Communication_Start(void);  
  
void App_Communication_ReceiveMsgByLoRa(void);  
#endif
```

6.2.5.2 App_Communication.c

```
#include "App_Communication.h"  
  
/**  
 * @description: 启动通信模块  
 */
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
void App_Communication_Start(void)
{
    /*1. 初始化 LoRa 模组 */
    Inf_LoRa_Init();

    /* 2. 初始化 W5500 模块 */
    Inf_W5500_Init();
}

/**
 * @description: 通过 LoRa 获取信息
 * 建议放入到 While 循环中。 当 LoRa 收到数据，就会执行相应的回调函数
 */
void App_Communication_ReceiveMsgByLoRa(void)
{
    Inf_LoRa_StartPollTask();
}

/* 覆写 LoRa 发送成功的回调函数 */
void LoRa_TransmitSuccCallBack(void)
{
}

/* 覆写 LoRa 接收成功的回调函数 */
void LoRa_ReceiveSuccCallBack(uint8_t *buffer, uint8_t length)
{
    debug_printfln("LoRa 收到数据");
    Inf_W5500_SendTcpData(buffer, length);
}
```

6.3 main.c

```
#include "main.h"
#include "spi.h"
#include "usart.h"
```

更多 [Java](#) -[大数据](#) -[前端](#) -[python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
#include "gpio.h"
#include "Common_Debug.h"
#include "App_Communication.h"
void SystemClock_Config(void);
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_SPI1_Init();
    MX_USART1_UART_Init();
    MX_SPI2_Init();
    /* 0. 初始化调试模块 */
    debug_init();
    debug_printfln("尚硅谷宠物定位项目之接收器...");

    /* 1. 启动通信模块 */
    debug_printfln("开启通信模块...");
    App_Communication_Start();
    while(1)
    {
        /* 1. 使用 lora 接收数据 */
        App_Communication_ReceiveMsgByLoRa();
    }
}
```